Journal of Food Measurement and Characterization (2025) 19:8469–8491
https://doi.org/10.1007/s11694-025-03559-1

ORIGINAL PAPER

# Real-time lightweight strawberry ripeness detection framework based on YOLO11 deployed on edge computing platform

**Yang Gan[1] · Xuefeng Ren[1] · Huan Liu[1] · Yongming Chen[1] · Ping Lin[1]**

Received: 25 January 2025 / Accepted: 4 August 2025 / Published online: 12 August 2025


## Abstract

Lightweight object detection algorithms that maintain a balance between speed and accuracy are essential for strawberry-picking robots. Strawberry fruit detection, however, is often challenged by occlusions from stems and leaves, overlapping fruits, and complex background interference. In addition, most existing ripeness detection methods rely on high-performance computing platforms, which limits their practicality in field applications. To address these issues, a lightweight real-time strawberry ripeness detection algorithm, SR-YOLO, is proposed based on the YOLO11 framework. A compact module, termed the Pela block, is designed using partial convolution (PConv) to reduce parameter count and memory access, while simultaneously suppressing background interference. A lightweight localization-based asymptotic feature pyramid network (LAFPN) is incorporated to enhance multi-scale detection performance. Moreover, a novel Focaler-PIoU loss function is introduced to direct the model toward anchor boxes of moderate quality, thereby mitigating the difficulty imbalance among strawberry samples. A large-scale, complex dataset is constructed, comprising 14,300 images and over 65,000 annotated strawberry instances, to evaluate the model. Experimental results indicate that the enhanced model achieves a precision of 91.3% and an mAP50 of 92.1%, with improvements of 2.8% and 2.5% over the baseline, respectively. Compared to mainstream detection algorithms, the proposed model demonstrates superior accuracy while maintaining a reduced number of parameters and lower computational complexity. When deployed on a Jetson AGX Orin embedded edge device with FP16 quantization, it sustains a detection speed of 90.1 FPS. In conclusion, SR-YOLO offers accurate, real-time detection of strawberry ripeness across complex agricultural environments, providing practical value for the vision modules of autonomous strawberry-picking robots.

**Keywords** Strawberry · Ripeness detection · YOLO11 · Edge computing · Agricultural environment

## Introduction

Strawberries are widely cultivated around the world due to their appealing flavor and high nutritional content, which has earned them the reputation of being the"Queen of Fruits"[1]. Their economic value is substantial, and demand continues to grow globally, with China ranking as the top producer [2]. Despite their popularity, strawberries are delicate and highly prone to damage from pests and diseases throughout their growth cycle. Without timely harvesting, they can rapidly spoil, leading to reduced quality and financial losses [3]. Currently, harvesting is primarily done by hand, which is both labor-intensive and costly, and is often associated with inaccuracies such as missed or incorrect picks. Robotic strawberry pickers offer a promising alternative due to their agility and ability to perform ripeness evaluations in real time, even in complex farm environments. Integrating ripeness detection algorithms into these robotic systems can greatly enhance productivity and lower labor demands. However, the limited memory and computing capabilities of edge devices present challenges for deploying such algorithms efficiently [4]. Moreover, the wide range of strawberry cultivars, the variability in fruit shape, and frequent occlusions introduce further complexity to the detection task. This underscores the necessity of lightweight detection models that can strike a balance between detection precision and speed, making them suitable for embedded applications in automated strawberry harvesting systems.

✉ Ping Lin
binglvcha007@126.com

[1] School of Electrical Engineering and Automation, Hubei Normal University, No.11 Cihu Road, Huangshi, Hubei 435002, PR China

Object detection approaches for strawberries can generally be divided into traditional image processing methods and modern deep learning-based solutions. Classical techniques primarily utilize the visual features of strawberries—such as color, shape, and surface texture—for identification. One common practice involves converting images to various color spaces to highlight distinguishing chromatic characteristics, thereby aiding segmentation and object localization [5]. Contour extraction tools like the Canny operator are frequently used to delineate strawberry edges [6], which, when combined with shape-matching algorithms or geometric pattern analysis, facilitate reliable recognition. Texture descriptors also play a key role by analyzing surface patterns to support the classification process [7]. In addition, filtering by size can help exclude non-fruit objects and refine the accuracy of detection. While these traditional methods can perform well under controlled conditions, they often struggle in more challenging scenarios involving complex backgrounds, variable illumination, and overlapping fruit clusters.

In the past ten years, the rapid development of artificial intelligence technologies has significantly driven the application of deep learning-based object detection in strawberry-related tasks [8]. Among these, Convolutional Neural Networks (CNNs) have demonstrated outstanding capabilities in learning rich feature representations [9], thereby substantially boosting detection accuracy in strawberry identification scenarios. Compared with traditional detection strategies, deep learning approaches not only offer greater precision but also exhibit superior adaptability in complex agricultural environments. Consequently, they have gradually become the dominant solutions for fruit detection tasks. Within the deep learning paradigm, object detection algorithms are typically categorized into two main classes: single-stage and two-stage models. For agricultural contexts, especially when targeting deployment on resource-constrained edge devices, single-stage detectors are often preferred due to their streamlined architecture, low latency, and reduced computational burden. These features make them well-suited for meeting the efficiency and mobility requirements of smart agricultural systems. As a result, significant research efforts have been directed toward the development and refinement of lightweight single-stage detection models optimized for real-time operation on portable platforms. For instance, Ji et al. [10] designed an apple detection model based on YOLOX, which achieved 26.3 frames per second (FPS) after TensorRT optimization on the Jetson Nano. Yang et al. [11] proposed a YOLOv5-based method for waxberry detection, successfully deploying it on a 2 GB Jetson Nano device with an FPS gain of 5.03 over the original YOLOv5. Similarly, Sun et al. [12] introduced a compact YOLOv5-based passion fruit detection algorithm that achieved a real-time speed of 11.25 FPS and had a lightweight model size of only 7.14 MB. These studies collectively highlight the practicality and efficiency of edge-based lightweight models for deployment in automated fruit-picking systems.

More recently, single-stage detectors have gained traction in real-time fruit harvesting applications owing to their rapid inference and low resource requirements. For example, Wang et al. [13] developed DSE-YOLO for strawberry detection, incorporating a Double Enhanced Mean Squared Error (DEMSE) loss function to better handle small fruit detection and foreground-foreground class imbalance. Pang et al. [14] presented an improved YOLOv5 model for assessing strawberry maturity, integrating a Depth-Mixed Multiscale Deformable Convolution (Ms-MDconv) structure to enhance multi-scale feature extraction. Meanwhile, Yang et al. [15] proposed a lightweight YOLOv8s-based maturity recognition model that leverages an LW-Swin Transformer module with multi-head self-attention to improve generalization. Their model achieved a precision of 94.4%, while utilizing only 51.93% of the original parameters and delivering a detection speed of 19.23 FPS.

Although these advancements contribute significantly to the detection and ripeness classification of strawberries, their performance on edge devices remains under-explored. Moreover, practical deployment still faces several unresolved issues. Firstly, in real-world agricultural scenes, strawberries are frequently occluded or surrounded by complex backgrounds, often resulting in missed or incorrect detections. Secondly, heavy occlusion can obscure vital features, impairing the model's ability to localize fruits accurately. Thirdly, many existing models are computationally expensive and parameter-heavy, making them unsuitable for high-speed edge-based processing. To overcome these challenges, we introduce SR-YOLO, a lightweight and high-precision strawberry ripeness detection algorithm based on the YOLO11n architecture. Designed for deployment on edge devices, SR-YOLO delivers accurate localization, efficient real-time detection, and enhanced classification performance. By addressing the trade-off between speed and accuracy, it enables reliable strawberry maturity assessment under complex environmental conditions and supports intelligent automation in fruit harvesting.

## Datasets preparation

### Acquisition of datasets

To enhance the model's generalization across varying cultivation conditions, four large-scale and diverse strawberry image datasets were constructed, ensuring that no duplicate images were included to maintain both accuracy and data integrity in the ripeness detection task.

(1) StrawDI_Db1 strawberry dataset: This publicly available dataset originates from 20 plantations spanning approximately 150 hectares in Huelva, Spain. Images were captured throughout the harvesting season under realistic field conditions. From this dataset, a subset of 2,500 images was randomly chosen for our study.
(2) Straw_Rf strawberry dataset: A total of 1,200 images were collected from the Roboflow platform (https://roboflow.com/), primarily sourced from the Strawberry-Detection and Strawberry-Detection2 collections.
(3) Straw_Sc strawberry dataset: This dataset comprises 1,800 images documenting different growth stages of strawberries cultivated in-house. The data were captured using various intelligent imaging devices under both indoor and outdoor conditions, spanning time periods from 6:00 AM to 6:00 PM, and covering multiple weather scenarios such as sunny and cloudy days. To increase diversity and minimize redundancy caused by overlapping fields of view, images were taken from multiple viewpoints and distances ranging from 0.2 to 2 m.
(4) LSDS strawberry dataset: This dataset was curated by combining images from Straw_Sc, StrawDI_Db1, and Straw_Rf, resulting in a total of 5,500 images.

In this work, the LSDS dataset was selected for ripeness classification due to its broad coverage of diverse environments and the variability in fruit appearance, providing a comprehensive representation of real-world strawberry growth conditions. Its versatility makes it highly suitable for ripeness detection tasks across different agricultural settings. Figure 1 presents some representative strawberry samples from the LSDS dataset. Additionally, we adopted ripeness classification standards from the literature [16] and classified strawberries into three maturity stages: Raw, Semi-ripe, and Ripe. The with the following criteria: Raw: Strawberries exhibit a pure white or green appearance; Semi-ripe: Strawberries range from pink to light red, covering 0% to 80% of the surface; Ripe: Strawberries turn dark red, with over 80% of the surface covered.


**Fig. 1** Captured representative strawberry fruit pictures from the diverse complex agricultural scenes stored in the LSDS dataset

## Augmentation of datasets

To improve dataset variability and better simulate the complexity of natural strawberry growth conditions, we applied a range of geometric and photometric transformations as part of our data augmentation strategy. Leveraging the Albumentations image augmentation library (https://github.com/albumentations-team/albumentations), various enhancement techniques were employed to enrich the dataset and increase its generalizability. As illustrated in Fig. 2, the original strawberry images were expanded by randomly applying four categories of augmentation operations—covering both spatial geometry and color attributes—to emulate the challenging and diverse environments encountered during real-world harvesting scenarios. To preserve the realism and biological plausibility of the augmented data, augmentation parameters were constrained based on empirical observations. Specifically, image rotation was confined to a narrow range of 0° to 15°, preventing excessive distortion while still capturing perspective shifts typically seen in field photography. Similarly, brightness alterations were limited to a $\pm 0.5$ range to avoid unnatural lighting artifacts, thus ensuring consistency with realistic agricultural lighting conditions. These constraints allowed the augmented samples to maintain visual fidelity with authentic strawberry appearances, thereby supporting robust feature learning. By randomly combining geometric and chromatic variations, the augmented dataset not only better reflects the diversity inherent in actual cultivation settings but also significantly improves the robustness and generalization capacity of the detection model. This approach enhances the model's ability to adapt to unpredictable conditions during automated strawberry harvesting tasks.

To prevent data leakage and ensure the statistical independence of training, validation, and testing samples, we initially divided the four constructed datasets into training, validation, and test subsets using a 6:2:2 split ratio. Importantly, data augmentation techniques were applied only to the training and validation subsets to enhance data diversity, while the test set remained unaltered to preserve evaluation integrity. In the original LSDS dataset, the distribution of annotated strawberry instances across the three ripeness categories included 7,384 raw, 4,288 semi-ripe, and 14,062 ripe strawberries. Following augmentation, the number of images in the dataset increased to 14,300. As summarized in Table 1, the post-augmentation instance counts for each

**Table 1** Instance distribution of raw, semi-ripe, and ripe strawberries on the LSDS dataset

| Types | Instances | Train | Validation | Test |
|---|---|---|---|---|
| Raw | 19198 | 11528 | 6198 | 1472 |
| Semi-ripe | 11148 | 6672 | 3613 | 863 |
| Ripe | 36567 | 21953 | 11780 | 2834 |
| Total | 66913 | 40153 | 21591 | 5169 |

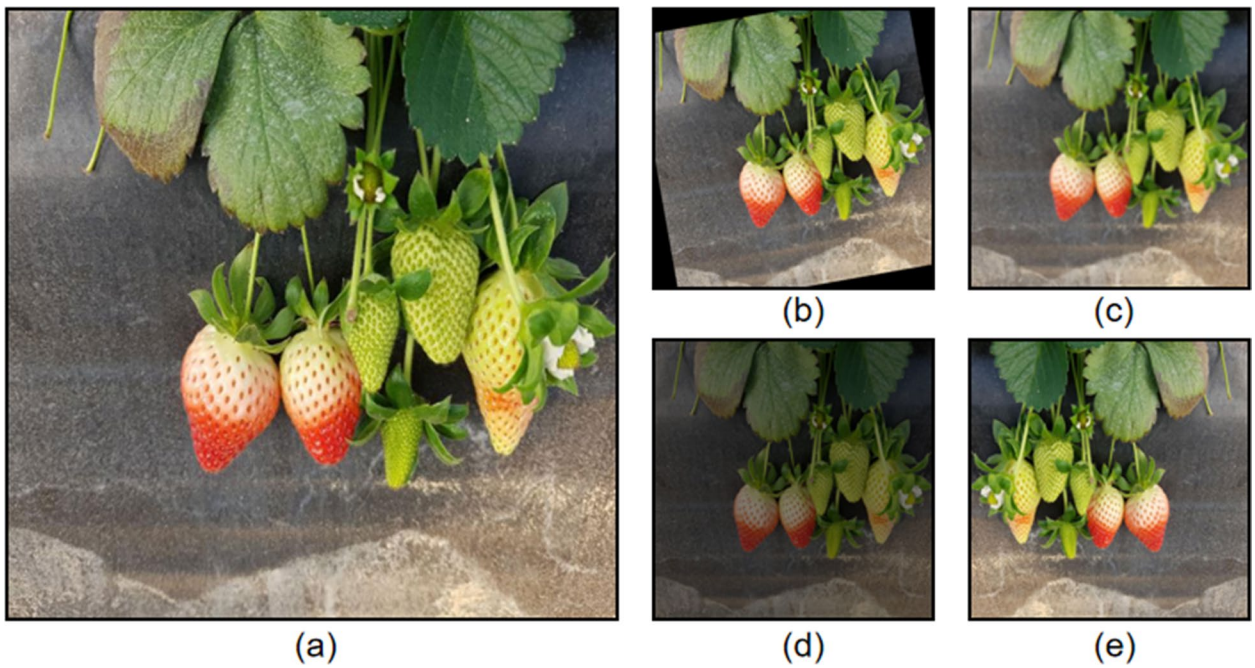


**Fig. 2** The raw cultivated strawberry image **a** is preprocessed by four different distinct data augmentation techniques: **b** Rotation, **c** Blur, **d** Brightness adjustment, **e** Horizontal flip by making use of Albumentations library, respectively

ripeness level expanded to 19,198 raw, 11,148 semi-ripe, and 36,567 ripe strawberries, respectively. These enriched distributions provide a more balanced and comprehensive training foundation for developing robust maturity classification models.

## Methodologies

In this study, a comprehensive improvement of the YOLO11n architecture is proposed, with successful deployment achieved on the NVIDIA Jetson AGX Orin platform to enable accurate and real-time detection of strawberries at different stages of ripeness. The overall workflow of the research is depicted in Fig. 3, which outlines key stages including dataset acquisition, data augmentation, model optimization, and edge deployment. Initially, the four constructed strawberry datasets were partitioned into training, validation, and testing subsets. To enhance sample diversity and model generalization, various data augmentation strategies were applied to the training and validation sets. The refined YOLO11n framework was then embedded with the proposed enhancements, followed by iterative rounds of training and validation to fine-tune the model and derive optimal hyperparameters and weight configurations. For real-world application, the trained model was deployed to the Jetson AGX Orin edge device. The PyTorch-based model was first exported to ONNX format and then converted into a TensorRT inference engine. To accelerate performance while maintaining detection precision, FP16 quantization was utilized. This deployment strategy enabled efficient and accurate maturity detection of strawberries under real-time field conditions.

### Overview of YOLO11

YOLO11 is an innovative model introduced by the Ultralytics development team (https://github.com/ultralytics/ultralytics), built upon YOLOv8, with several new features and improvements that enhance the model's detection performance and flexibility. This comprehensive model integrates multiple functionalities, including object detection, tracking, segmentation, and classification, among others. Compared to YOLOv8, YOLO11 offers significant advancements in the following three key aspects:

(1) Replacement of the C2f module with the enhanced C3k2 module: As shown in Fig. 4(a), the C3k2 builds upon the traditional C3 structure by incorporating deformable convolution and channel separation strategies. This modification significantly strengthens feature extraction in the backbone and enhances feature fusion

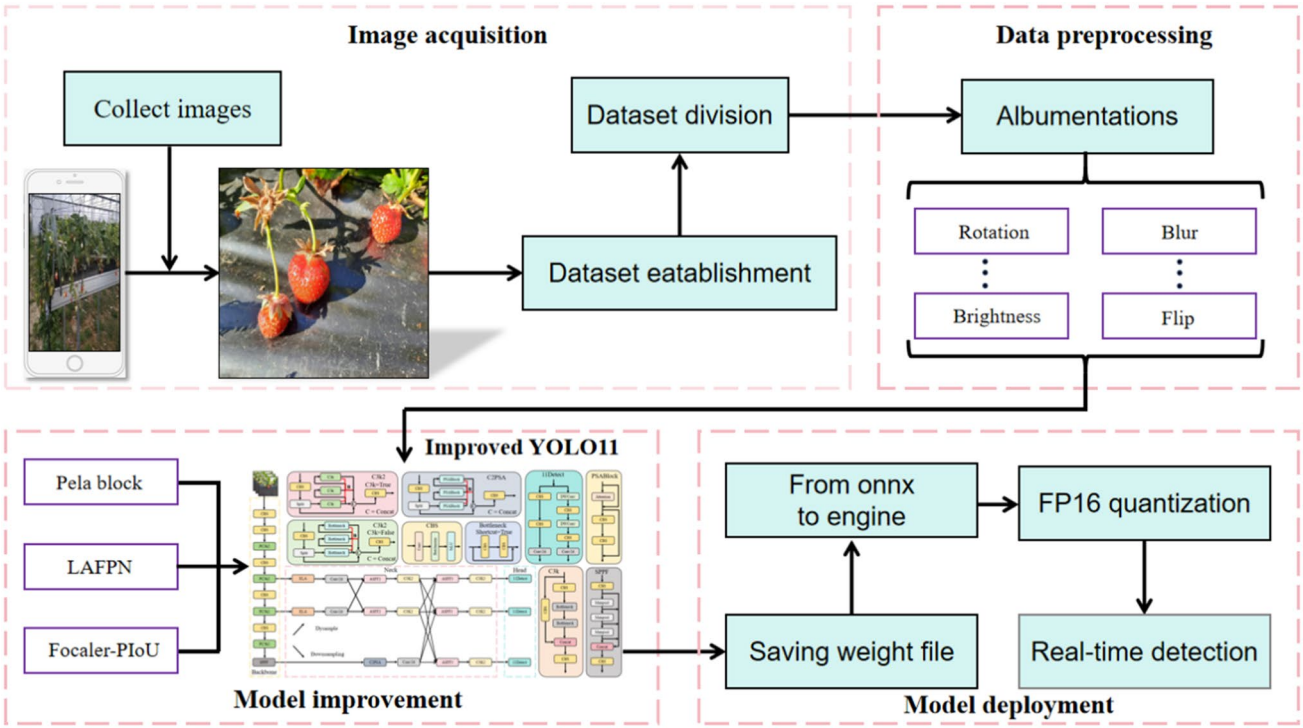


**Fig. 3** Flow chart of the deployment of the proposed architecture for real-time and accurate identification of strawberries with different maturity levels in complex agricultural environments on edge computing devices. There are four components of image acquisition, data preprocessing, improved YOLO11 models, and model deployment

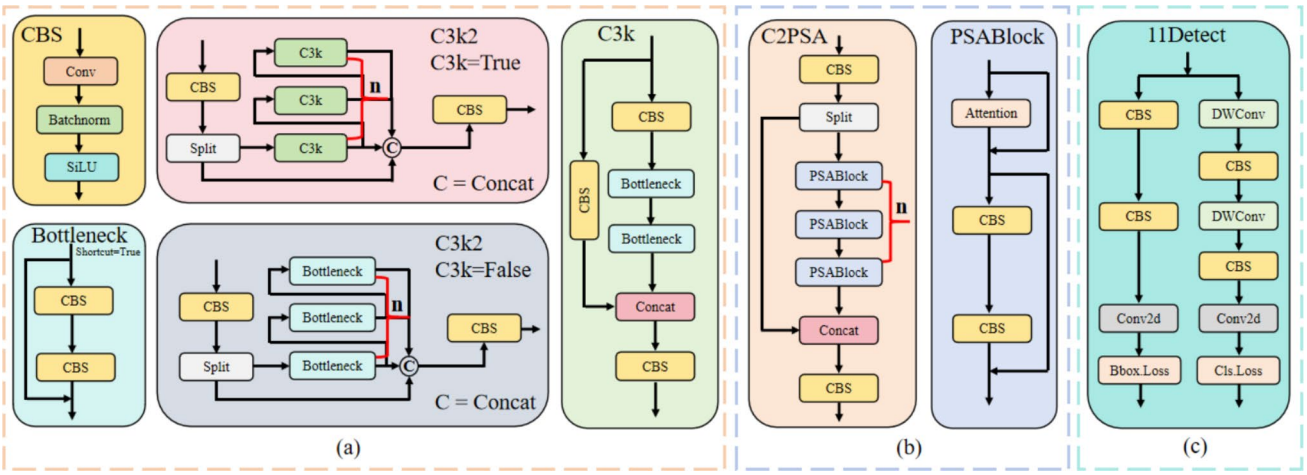


**Fig. 4** Key improvements in YOLO11 architecture

in the neck, thereby improving overall detection accuracy.

(2) Introduction of the C2PSA module after the PSA module: As depicted in Fig. 4(b). this module integrates the Cross-Stage Partial (CSP) structure with the Partial Self-Attention (PSA) mechanism. The design enhances multi-scale feature extraction, allowing the model to more effectively capture and utilize features across different spatial resolutions.

(3) In the classification head, YOLO11 employs depth-wise separable convolutions (shown in Fig. 4(c)) to reduce redundant computations, improving the model's computational efficiency. This approach not only reduces the model's parameter count but also facilitates deployment on resource-constrained embedded devices.

## Improved strategy of YOLO11 model

We present SR-YOLO, a lightweight and real-time strawberry ripeness detection framework built upon the YOLO11n architecture. The proposed method achieves high detection accuracy across various ripeness stages while preserving the computational efficiency and compact design of the base model, making it ideal for deployment in resource-constrained agricultural scenarios. The complete architecture of SR-YOLO is depicted in Fig. 5. To improve the backbone's feature extraction capability, particularly for visually ambiguous immature strawberries, we replace the conventional bottleneck module within the original C3k2 block with a Pela block. This structural substitution enhances the network's ability to distinguish small or background-blending targets while simultaneously reducing computational burden. For the neck component, we integrate the Localization-based Asymptotic Feature Pyramid Network (LAFPN). This module is designed to enhance multi-scale feature fusion by capturing both fine-grained local details and broader contextual semantics, which are essential for reliable classification in complex natural environments. In the prediction head, we replace the traditional CIoU loss with the Focaler-PIoU loss function, which is tailored to address the imbalanced detection difficulty across different ripeness levels. This adaptive loss mechanism improves the precision and robustness of bounding box regression by dynamically adjusting the contribution of hard-to-detect samples. With these targeted modifications, the SR-YOLO framework delivers high-performance strawberry detection across all maturity categories while maintaining real-time inference speed. Subsequent sections provide a detailed description of each component and the techniques used to construct the proposed model.

## Lightweight compact module based on PConv operator

The original C3k2 module in YOLO11n comprises a combination of C3k and C2f blocks, designed to facilitate the transmission of multi-level gradient signals. This architectural configuration alleviates issues such as gradient vanishing and explosion [17], thereby enhancing the network's capacity for hierarchical feature extraction and improving overall detection performance. However, the frequent use of standard convolutional operations within the C3k2 structure may result in the extraction of redundant or non-discriminative features. This can increase computational overhead and degrade model efficiency and precision. To overcome these limitations, we incorporate the Partial Convolution (PConv) operator [18], which leverages spatial redundancy in the input feature maps. PConv selectively processes spatial regions, allowing important channel information to be preserved while significantly reducing the number of parameters and memory access costs. This promotes efficient spatial

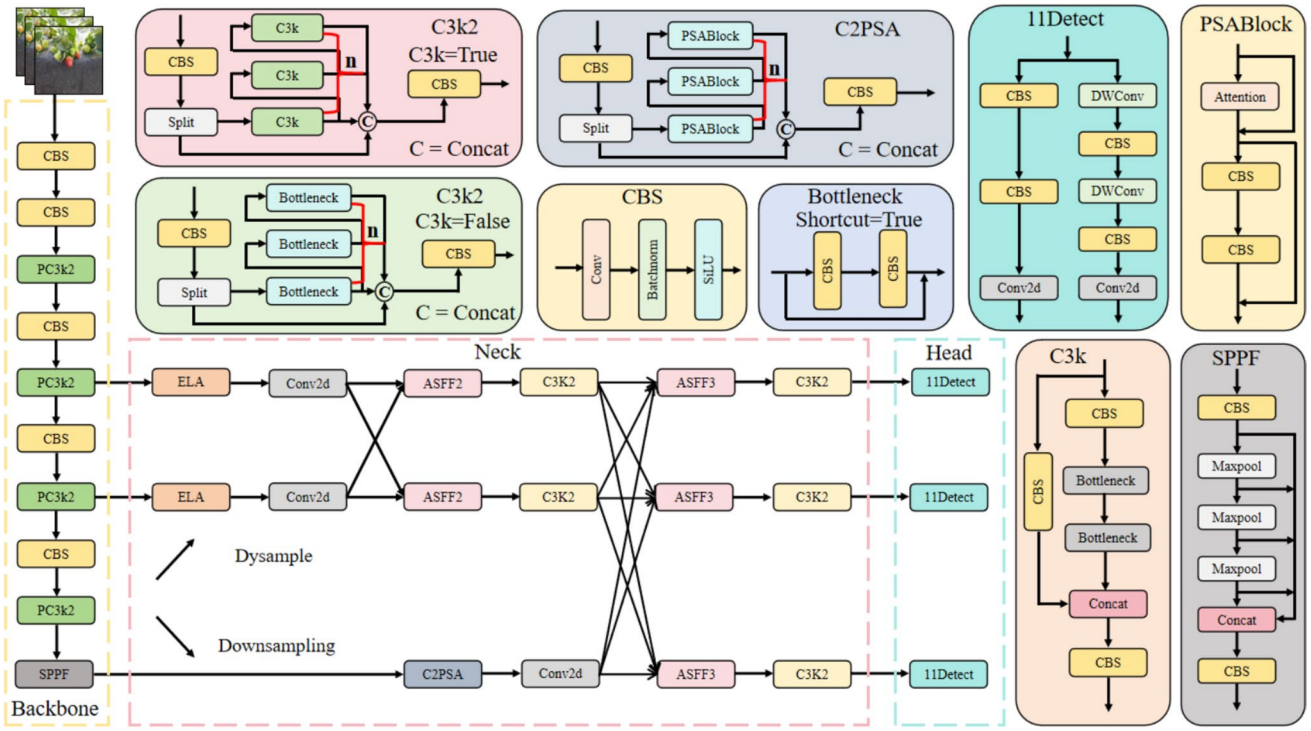


**Fig. 5** Overall proposed lightweight framework of SR-YOLO adapt for real-time and accurate identification of strawberries at different maturity levels under complex agricultural circumstances

feature encoding and minimizes unnecessary computation. Building on this, we propose a lightweight module termed the Pela block, constructed using the PConv operator, as shown in Fig. 6b. The Pela block replaces the conventional bottleneck unit in the C3k2 structure, leading to the formation of a redesigned backbone component referred to as the PC3k2 module. This revised module enhances spatial sensitivity and positional representation while maintaining a compact architecture. Additionally, it effectively suppresses background noise and visually similar non-target features, which often hinder accurate strawberry identification in complex agricultural environments. The incorporation of the PC3k2 module contributes to lower false detection rates and improved model robustness across different ripeness stages, thereby enabling more reliable and efficient detection performance in real-world scenarios.

By integrating the PConv operator with the efficient local attention (ELA) [19] module, we successfully constructed the Pela operator, as depicted in Fig. 6a. This operator applies filters and the ELA module selectively to a subset of input channels, leaving the other channels intact. This method efficiently reduces unnecessary computations and memory accesses, improving the utilization of spatial features. We set $C_p$ to 1/4 as the default value, and the specific calculation process of the Pela operator is described as follows:

$$X_{main}, X_{res} = Split(X, [C_p, C - C_p]) \tag{1}$$

where $X$ refers to the input feature map consisting of $C$ channels, $X_{main}$ representes to the input feature map consisting of $C_p$ channels, and $X_{res}$ is the input feature map with $C - C_p$ channels, *Split* represents the division of the channel count.

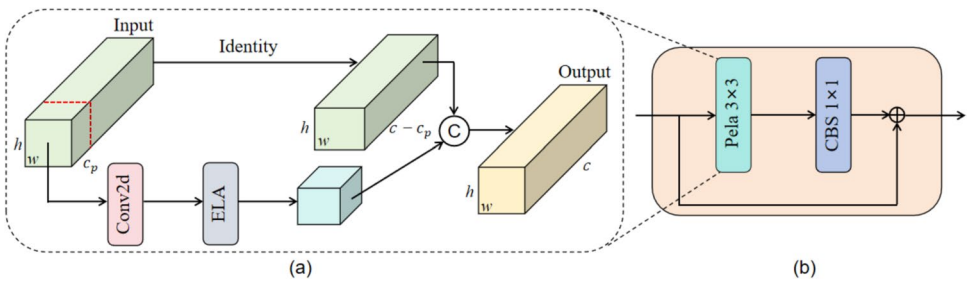


**Fig. 6** Pela block and its core elements: **a** Pela operator, **b** Pela block

$$X_{ela} = ELA(Conv2d(X_{main})) \tag{2}$$

where *Conv2d* denotes the standard convolution with a kernel size of 3. The operations *Conv2d* and *ELA* are applied only to $X_{main}$, without affecting the other channels. Consequently, this method reduces the computational overhead and memory access requirements, enhancing the extraction and use of spatial features.

$$Y = Concat(X_{ela}, X_{res}) \tag{3}$$

where *Concat* represents the concatenation operation, and $Y$ is the final output.

It is evident that the integration of the Pela operator substantially improves the network's computational efficiency by enabling the targeted extraction of key positional features from strawberry images. In contrast to conventional convolutional mechanisms, the Pela operator enhances local feature discrimination while simultaneously accelerating inference. This dual benefit strengthens the model's capacity to distinguish foreground targets from background clutter, which is particularly critical in visually complex agricultural environments. As a result, the operator contributes to more compact and expressive feature representations, thereby enabling accurate localization and classification of strawberries at varying ripeness levels. These improvements provide a solid foundation for deploying real-time ripeness detection systems in practical field applications.

## ELA(efficient local attention)

In the task of detecting the ripeness of strawberries, attention mechanisms enable the model to more precisely identify strawberries at various ripeness stages [20]. To this end, we introduce an efficient local attention (ELA) method, and its structure is depicted in Fig. 7. To speed up the processing speed of the ELA module, we use HardSigmoid [21] instead of the original Sigmoid activation function. Firstly, for a given input feature map $X \in R^{H\times W\times C}$,we apply two spatial range average pooling on each channel. The following formulas are used to calculate the positional information in the horizontal and vertical directions:

$$Z_c^h(h) = \frac{1}{H}\sum_{0\le i\le H} x_c(h,i) \tag{4}$$

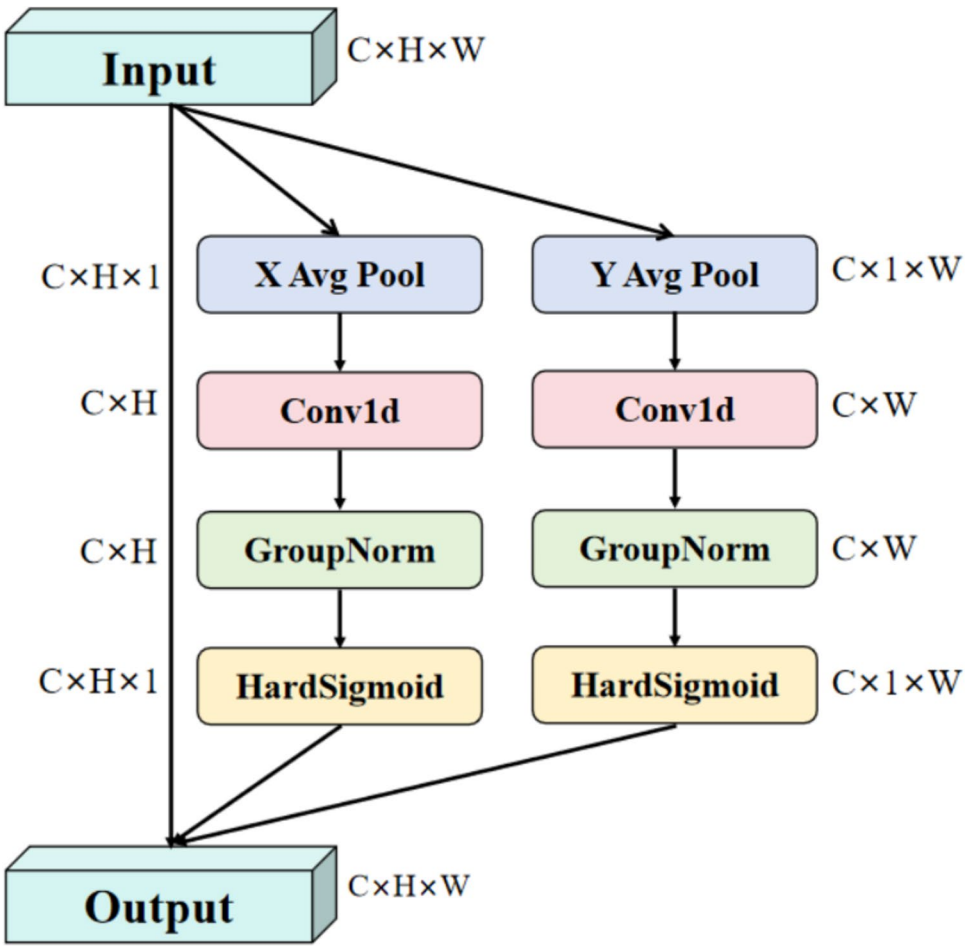


**Fig. 7** Structure of the efficient local attention module

$$z_c^w(w) = \frac{1}{W} \sum_{0 \le j \le W} x_c(j, w) \tag{5}$$

where $i$ and $j$ serve as positional markers to identify specific elements within the matrix. Next, a 1D convolution is used to improve positional information along the horizontal and vertical axes. Finally, the positional attention representation is generated through group normalization $\sigma$ and a nonlinear activation function $\sigma$. The specific expressions are shown as follows:

$$y^h = \sigma(G_n F_h(z_h)) \tag{6}$$

$$y^w = \sigma(G_n F_w(z_w)) \tag{7}$$

where the 1D convolution is represented by $F_h$ and $F_w$, while the positional attention in the horizontal and vertical directions is denoted by $y^h$ and $y^h$ respectively. The final output of the ELA module is then computed as follows:

$$Y = x_c \times y^h \times y^w \tag{8}$$

The incorporation of the ELA module enables the model to effectively extract fine-grained local features across different channels while concurrently capturing long-range spatial dependencies. This dual capability allows for direction-sensitive and accurate localization of strawberry instances, supporting the learning of richer and more comprehensive feature representations. As a result, the model exhibits improved recognition performance across varying ripeness levels, even in visually complex or dynamic environments. Furthermore, this enhancement contributes to greater robustness and generalization, making the model more reliable for deployment in real-world strawberry detection scenarios.

## Localization-based asymptotic feature pyramid network

YOLO11 largely inherits the neck network structure from YOLOv10, utilizing the PAN-FPN framework [22] to enable multi-scale feature fusion. While effective in aggregating features across layers, this hierarchical path aggregation can introduce notable computational overhead [23]. Furthermore, the detection of small-scale targets remains a persistent challenge, which may limit the model's responsiveness in real-time strawberry detection tasks. In addition, the reliance on standard nearest-neighbor interpolation for upsampling may cause important contextual details to be lost, particularly from adjacent pixels, thus compromising the precision of fine-grained feature representation [24]. To address these issues, we propose a lightweight Localization-based Asymptotic Feature Pyramid Network (LAFPN), as illustrated in Fig. 8. LAFPN is mainly divided into two stages: (1) feature localization; (2) feature progressive fusion. Firstly, high-level features rich in semantic information are processed through the C2PSA module, while low-level features with higher resolution undergo feature filtering through the ELA module. Then, ordinary $1 \times 1$ convolutions are used on feature maps of various scales to modify the channel count. Finally, through progressive fusion strategies, different feature maps are effectively combined, preserving rich semantic information

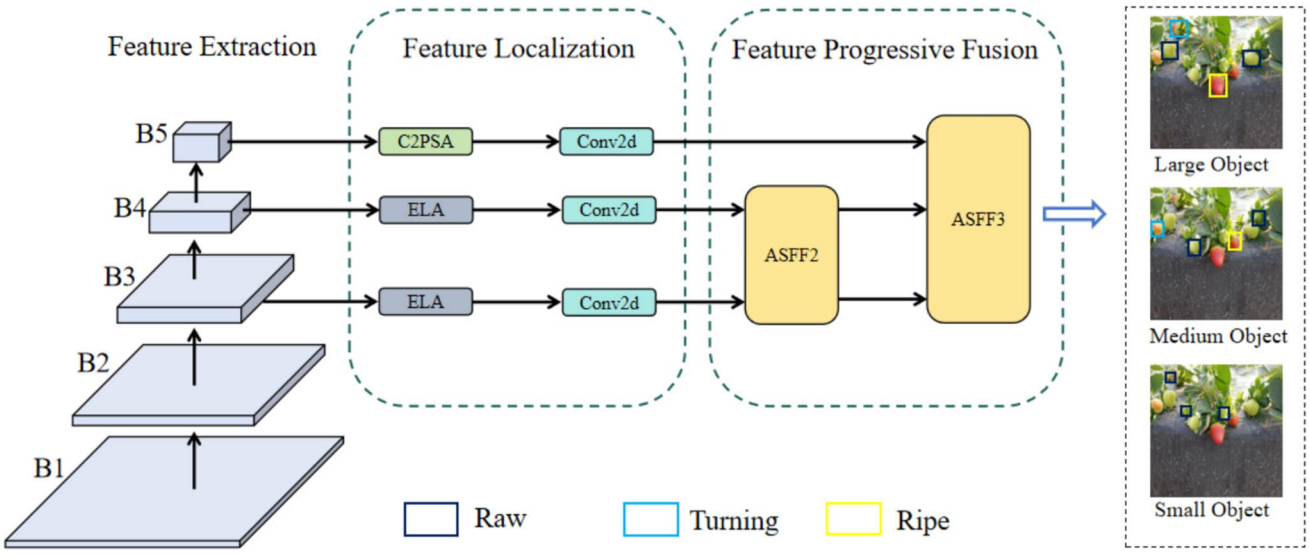


**Fig. 8** Framework of localization-based asymptotic feature pyramid network comprises two parts: feature localization and feature progressive fusion

while ensuring accurate positional information, reducing conflicts between different target information, thereby improving the model's capability to identify strawberries at various ripeness stages in challenging environments.

### Feature localization

In the feature localization stage, the ELA module, in conjunction with conventional convolutional layers, plays a crucial role in enhancing spatial perception. By integrating coordinate-aware mechanisms, the ELA module enables the network to encode spatial positional relationships directly into the feature learning process. This facilitates a more precise understanding of the spatial layout of strawberry targets within an image, thereby supporting more accurate classification across ripeness stages.

Moreover, through the computation of spatial attention weights, the model is capable of dynamically focusing on regions that are most relevant for ripeness estimation. This selective emphasis enhances the representational quality of discriminative features while simultaneously suppressing irrelevant background information, accommodating variations in fruit scale, and improving overall detection reliability. Concurrently, the C2PSA module refines high-level features by enriching semantic content, allowing the model to capture broader contextual cues critical for interpreting global scene information. The synergy between the ELA module and conventional convolutions allows the model to more effectively isolate and highlight salient regions extracted from the backbone network. As a result, the system demonstrates improved sensitivity to visual attributes essential for differentiating ripeness stages, contributing to enhanced robustness and accuracy in challenging detection scenarios.

### Feature progressive fusion

In the progressive feature fusion stage, we incorporate the Asymptotic Feature Pyramid Network (AFPN) [25], which works in tandem with the retained C3k2 module to facilitate rapid integration of multi-scale features. The C3k2 structure contributes to efficient feature blending across different resolution levels, while maintaining the integrity of critical information.

To further enhance fusion efficiency, we introduce the DySample operator, a lightweight and adaptive upsampling mechanism that enables more refined spatial resolution adjustment with minimal computational overhead. By combining DySample with Adaptive Spatial Feature Fusion (ASFF) [26], the model achieves effective alignment and fusion of spatial and semantic information across scales. This approach not only preserves fine-grained positional cues and high-level semantic richness but also significantly lowers the resource demands typically associated with multi-scale processing, thereby improving real-time detection performance. As illustrated in Fig. 9, the fusion process begins by aligning the spatial dimensions of neighboring feature maps through either upsampling or downsampling operations. Once harmonized, adjacent layers are progressively fused, enabling seamless information propagation across the feature hierarchy. This design ensures that the network can effectively integrate context from both local and global scales, ultimately enhancing its ability to detect strawberries with varying visual characteristics in complex environments. Subsequently, feature adaptive spatial fusion is used, with a greater focus on critical layers, and the feature fusion between two layers is demonstrated as follows:

$$y_{ij}^{l} = \alpha_{ij}^{l} \cdot x_{ij}^{1\to l} + \beta_{ij}^{l} \cdot x_{ij}^{2\to l} \tag{9}$$

where $\alpha_{ij}^{l}$ and $\beta_{ij}^{l}$ denote the spatial weights used in the linear concatenation of features from two distinct levels,

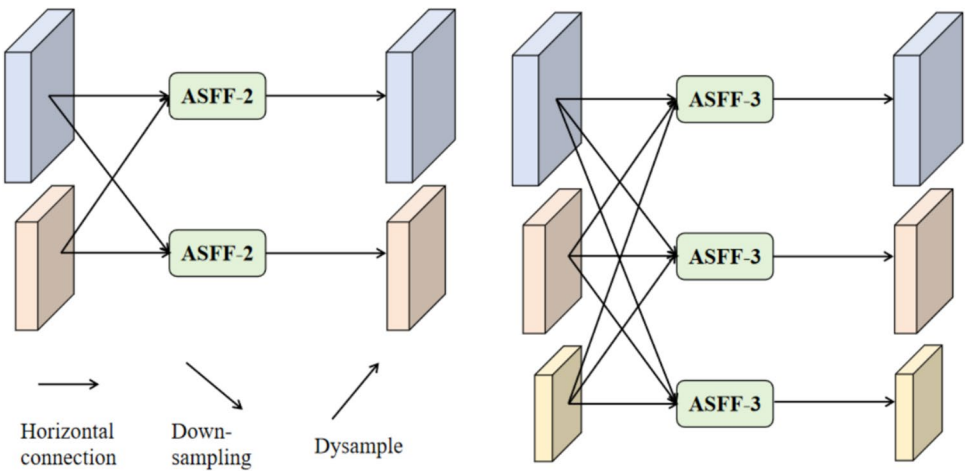


**Fig. 9** Adaptive spatial fusion operation for feature maps at different levels

constrained by condition $\alpha_{ij}^{l} \bullet \alpha_{ij}^{l} + \beta_{ij}^{l} + \gamma_{ij}^{l} = 1$. $x_{ij}^{1 \to l}$ denotes the feature vector from the $n-th$ layer to the $n-th$ layer at position $(x, y)$. $\gamma_{ij}^{l}$ represents the new feature vector derived from the adaptive spatial fusion of multi-level features. Similarly, the combination of three-level feature vectors is presented as follows::

$$y_{ij}^{l} = \alpha_{ij}^{l} \cdot x_{ij}^{1 \to l} + \beta_{ij}^{l} \cdot x_{ij}^{2 \to l} + \gamma_{ij}^{l} \cdot x_{ij}^{3 \to l} \tag{10}$$

where $\alpha_{ij}^{l}$, $\beta_{ij}^{l}$, and $y_{ij}^{l}$ correspond to the spatial weights of the three-level features in the $l$ layer, which are constrained by condition $\alpha_{ij}^{l} + \beta_{ij}^{l} + \gamma_{ij}^{l} = 1$.

During the image sampling process, we employ a $3 \times 3$ convolution layer with a stride of 2 to downsample the feature maps, compressing the information and reducing computational load. Simultaneously, we introduce the DySample operator [27] to perform upsampling, which focuses on recovering the resolution of the feature maps and improving their detail representation. Its structural framework is shown in Fig. 10. Unlike traditional dynamic convolution methods, the DySample operator relies on advanced point sampling technology. This technology first generates content-aware offsets for the input feature $X$ through a linear projection layer. These offsets accurately guide the subsequent sampling process, ensuring the precision and relevance of the resampling. After obtaining the offsets, we use bilinear interpolation to perform a meticulous resampling of the original input feature $X$. Specifically, the input feature $X$ enters the sampling point generator, which processes it into a set $S$ containing sampling information. Each item in this set represents a scale factor for upsampling and the corresponding $x$ and $y$ coordinates of the original feature $X$. Next, the grid sampling function uses the coordinate positions within set S to perform an appropriate upsampling operation on the input feature $X$, thereby generating a new feature map X'. The design philosophy of the DySample dynamic upsampler endows it with high flexibility and adaptability. Compared to traditional interpolation methods in YOLOv8, DySample not only achieves a significant improvement in model inference speed but also demonstrates a notable advantage in reducing the loss of feature details.

## Focaler-PIoU loss function

In strawberry ripeness detection tasks, occlusion by stems or leaves and the frequent occurrence of densely clustered fruits often result in missed detections and false positives [3].As such, designing an appropriate loss function is vital for enhancing the model's localization precision and overall detection reliability. Although the CIoU loss adopted in YOLO11 leverages geometric relationships between predicted and ground-truth bounding boxes to refine regression quality, it does not consider the varying difficulty levels across training samples. Likewise, the CIoU loss—despite integrating center distance and aspect ratio consistency—can yield overly punitive gradients in cases where initial spatial discrepancies between prediction and target are large, potentially hindering model convergence.

To overcome these drawbacks, we introduce a novel loss function termed Focaler-PIoU, which fuses the complementary strengths of Focaler-IoU and Powerful-IoU (PIoU). This hybrid formulation directs the optimization process toward harder-to-learn samples and medium-quality anchor boxes, thereby improving the model's ability to refine bounding box predictions in challenging detection settings. By adaptively emphasizing informative samples during training, the proposed Focaler-PIoU loss enhances both the accuracy and robustness of the regression process, particularly under complex environmental conditions involving occlusions and dense target distributions.

### PIoU

The Powerful Intersection over Union (PIoU) loss function [28] introduces a refined approach to bounding box regression by integrating a target-size-adaptive penalty term and a gradient modulation mechanism based on anchor box quality. This dual strategy is designed to enhance detection accuracy and model stability, particularly under challenging real-world conditions in agricultural environments. The target-size-adaptive penalty dynamically scales the regression penalty according to the size of the target object, allowing the model to better accommodate the inherent scale variability of strawberries, especially in cases involving extremely

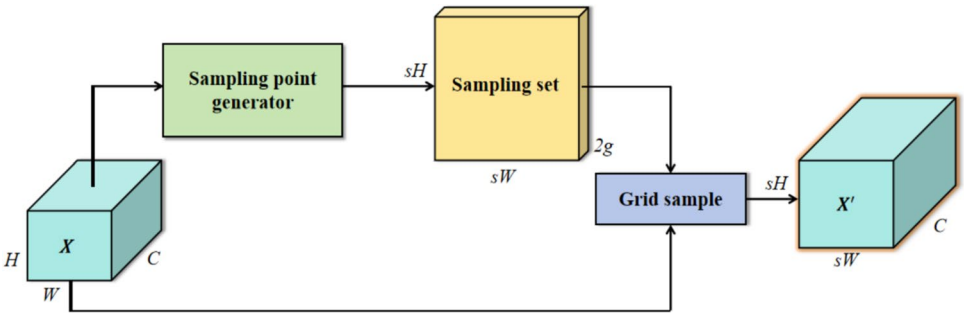


**Fig. 10** Structure of Dysample network for boosting the features and the inference speed by leveraging the advanced point sampling technology

small or large instances. Meanwhile, the gradient adjustment function, which evaluates anchor box quality during training, prioritizes learning from more reliable samples. This selective optimization process leads to more stable convergence and improved localization precision.

To further support real-time deployment, PIoU incorporates an efficient path-aware regression mechanism, which expedites convergence and enables faster and more accurate predictions—an essential feature for applications requiring immediate decision-making, such as robotic strawberry harvesting. Additionally, a non-monotonic attention weighting scheme is employed to strengthen the model's focus on medium-quality samples, which are often overlooked in traditional loss formulations. This enhances detection robustness across varying annotation quality and environmental complexity. The mathematical formulation and computational process of the PIoU loss function are described in detail as follows:

The calculation of IoU (Intersection over Union) for PIoU is as follows:

$$IoU = \frac{|b \cap b^{gt}|}{|b \cup b^{gt}|} \tag{11}$$

where $b$ denotes the predicted box, while $b^{gt}$ represents the ground truth box. The calculation of the overlap and non-overlap distances in the x-axis direction between the two bounding boxes for PIoU is as follows:

$$\begin{cases} dw1 = \left|\min(b1_{x2} - b1_{x1}) - \min(b2_{x2} - b2_{x1})\right| \\ dw2 = \left|\min(b1_{x2} - b1_{x1}) - \min(b2_{x2} - b2_{x1})\right| \end{cases} \tag{12}$$

where $dw1$ represents the absolute overlap distance along the x-axis between the first and second bounding boxes, and $dw2$ indicates the absolute non-overlap distance along the x-axis between the two bounding boxes. The calculation of the overlap and non-overlap distances in the y-axis direction between the two bounding boxes for PIoU is given as follows:

$$\begin{cases} dh1 = \left|\min(b1_{y2} - b1_{y1}) - \min(b2_{y2} - b2_{y1})\right| \\ dh2 = \left|\min(b1_{y2} - b1_{y1}) - \min(b2_{y2} - b2_{y1})\right| \end{cases} \tag{13}$$

where $dh1$ denotes the absolute overlap distance along the y-axis between the first and second bounding boxes, and $dh2$ represents the absolute non-overlap distance along the y-axis between them. The formula for calculating the penalty factor $p$ is provided as follows:

$$p = \frac{1}{4}\left(\frac{dw1 + dw2}{\left|w_{gt}\right|} + \frac{dh1 + dh2}{\left|h_{gt}\right|}\right) \tag{14}$$

where$w_{gt}$,$dw2$,$dh1$, and $dh2$ repressent the absolute distances between the corresponding sides of the predicted and target bounding boxes. Additionally, $w_{gt}$ and $h_{gt}$ represent the width and height of the target bounding box, respectively.The base loss of PIoU is presneted as follows:

$$L = 2 - IoU - e^{-p^2} \tag{15}$$

We integrated a non-monotonic attention function with the loss function $L$ to formulate a novel loss function $L_{PIoU}$. This novel loss function $L_{PIoU}$ is derived by augmenting $L$ with an attention layer, thereby enhancing the ability of the model to focus on medium to high-quality anchor boxes. This enhancement contributes to the improved performance of the object detector. The computational formulation is detailed as follows:

$$q = e^{-p} \tag{16}$$

$$u(x) = 3x \cdot e^{-x^2} \tag{17}$$

$$L_{PIoU} = u(\lambda q) \cdot L = 3 \cdot (\lambda q) \cdot e^{-(\lambda q)^2} \cdot L \tag{18}$$

where $u(\lambda q)$ denotes the attention function. We substitute the penalty factor $p$ with $q$, which assesses the quality of the anchor box. As $p$ increases, $q$ progressively decreases, suggesting a decline in anchor box quality. Meanwhile, $\lambda$ acts as a hyper parameter that controls the behavior of the attention function.

#### Focaler-IoU

The Focaler-IoU loss function [29] adjusts the relative weights according to the difficulty level of the samples, allowing the model to give more attention to challenging samples during training, such as strawberries that are occluded, too small in size, or interfered with by background colors. By this method, the model not only pays more attention to difficult-to-classify positive samples but also effectively improves its response to the imbalance between hard and easy samples by reducing the weights of easily recognizable samples. Focaler-IoU reconstructs the IoU loss through a linear interval mapping method, with the calculation formula as follows:

$$IoU^{focaler} = \begin{cases} 0, IoU < d \\ \frac{IoU-d}{u-d}, d << IoU << u \\ 1, IoU > u \end{cases} \tag{19}$$

where $IoU$ represents the ratio of the intersection to the union between the predicted bounding box and the ground truth box, which stands for the reconstructed Focaler-IoU. The values of variables $u$ and $d$ are constrained within the

range of [0,1]. By adjusting the values of $u$ and $d$, the model can effectively adjust for various regression samples, thereby improving its ability to distinguish between hard and easy samples. The loss function is defined as follows:

$$L_{Focaler-IoU} = 1 - IoU^{focaler} \tag{20}$$

#### Synergistic loss

By integrating the Focaler-IoU loss into the PIoU bounding box regression loss function, we construct the Focaler-PIoU loss function. The specific definition of this loss function is as follows:

$$L_{Focaler-PIoU} = L_{PIoU} + IoU - IoU^{Focaler} \tag{21}$$

The Focaler-PIoU loss function combines the emphasis on focusing on medium-quality anchor boxes from PIoU with the attention to difficult samples from Focaler-IoU, effectively overcoming the limitations of traditional IoU loss functions and substantially enhancing the detection accuracy of strawberry fruit targets.

## Results and discussion

### Experiment setup

All experimental procedures in this study were conducted on a Linux-based server environment, equipped with Python 3.10.13, PyTorch 2.2.1, and CUDA 12.1 for GPU acceleration. The computational infrastructure comprised an Intel® Xeon® Platinum 8352 V processor operating at 2.10 GHz, supported by 24.0 GB of system RAM, and two NVIDIA GeForce RTX 4090 GPUs, ensuring sufficient computational power for high-throughput model training. To fully exploit the capabilities of the proposed SR-YOLO framework, a set of carefully selected training hyperparameters was adopted, as detailed in Table 2. The momentum coefficient was set at 0.937, with a weight decay of 0.0005, and an initial learning rate of 0.01. A step decay schedule was applied, reducing the learning rate by 50% every 10 epochs to facilitate gradual convergence and refined parameter updates. All training images were uniformly resized to 640 × 640 pixels. The training process was carried out over 300 epochs using the stochastic gradient descent (SGD) optimizer. To mitigate overfitting and enhance training efficiency, an early stopping criterion was employed: if the model's accuracy did not improve over 50 consecutive epochs, training was halted automatically. This strategy ensures the preservation of the best-performing model checkpoint while minimizing unnecessary computational resource usage.

In the context of developing vision systems for automated strawberry harvesting robots, the ability to accurately and rapidly localize fruits across various ripeness stages requires the integration of both efficient detection algorithms and high-performance embedded hardware. To meet these requirements, the NVIDIA Jetson AGX Orin is adopted in this study as the edge AI computing platform, owing to its powerful processing capabilities and compact design, which make it well-suited for real-time deep learning inference in field conditions.

As summarized in Table 3, the Jetson AGX Orin delivers up to 275 trillion operations per second (TOPS) of AI performance, driven by its NVIDIA Ampere architecture GPU comprising 2048 CUDA cores and 64 Tensor Cores, enabling the execution of complex neural network models with low latency. The device is further equipped with a 12-core Arm Cortex-A78AE v8.2 64-bit CPU and 64 GB of LPDDR5 memory on a 256-bit bus, offering a peak memory

**Table 2** Summary of main parameters for the training our SR-YOLO detection model

| Parameter | Details | Parameter | Details |
|---|---|---|---|
| Epoch | 300 | Batch-size | 128 |
| Initial learning | 0.01 | Optimizer | SGD |
| Workers | 8 | Image-size | 640 × 640 |
| Weight_decay | 0.0005 | Momentum | 0.937 |

**Table 3** Specifications of NVIDIA Jetson AGX Orin edge computing equipment for our real-time strawberry object detection application

| Feature category | Details |
|---|---|
| AI Computing Performance | Up to 275 TOPS of AI performance |
| GPU | NVIDIA Ampere architecture |
| CPU | 12-core Arm Cortex-A78AE v8.2 64-bit CPU |
| Memory | 64 GB 256-bit LPDDR5 memory |
| Memory Bandwidth | Up to 204.8 GB/s |
| Storage | 64 GB eMMC 5.1 storage |
| Camera Interface | 16-channel MIPI CSI-2 connectors |
| Power Configuration | Adjustable power configuration from 15 to 60W |
| Expansion Interfaces | PCIe slot |

bandwidth of 204.8 GB/s, which facilitates high-throughput data processing essential for real-time applications.

One of the standout features of the Jetson AGX Orin is its dynamic power scaling capability, supporting operational power ranges from 15 to 60 W, allowing for adaptive energy consumption based on workload demands. This flexibility is especially advantageous for deployment in mobile or power-sensitive robotic platforms, as it ensures consistent performance while managing energy efficiency in variable operating environments. Overall, the Jetson AGX Orin presents a well-balanced and practical solution for deploying lightweight, high-accuracy object detection models—such as the proposed SR-YOLO—in real-world strawberry harvesting scenarios, where both inference speed and hardware efficiency are critical.

Finally, we deployed the optimized model on the Jetson AGX Orin, utilizing a USB color camera to capture real-time strawberry images. This configuration enabled the real-time detection of strawberries in practical settings, as demonstrated in Fig. 11.

## Evaluation metrics

In this study, we assess the model's performance using a wide range of metrics, including accuracy, speed, and complexity. To thoroughly assess detection accuracy, we focus on three key indicators: Precision (P), Recall (R), and Average Precision (AP). The formulas are outlined below:

$$P = \frac{TP}{TP + FP}; R = \frac{TP}{TP + FN} \tag{22}$$

where $TP$ denotes the number of samples correctly classified as positive, $TP$ denotes the number of samples incorrectly classified as positive, and $FN$ represents the number of samples incorrectly classified as negative.

$$AP = \int_0^1 P(R)dR; mAP = \frac{1}{m}\sum_{i=1}^{m} AP_i \tag{23}$$

where $AP$ is a comprehensive metric derived from the Precision-Recall (P-R) curve. m represents the number of IoU thresholds, and $AP_i$ is the $AP$ at the $i-th$ IoU threshold. In object detection tasks, $maP$ mis a crucial evaluation metric, as it assesses the model's overall performance under various detection conditions. Specifically, mAP50 refers to the $maP$ calculated at an IoU threshold of 0.50, primarily used to evaluate the model's detection capability under relatively loose overlap conditions. In contrast, mAP95 represents the average $maP$ calculated over an IoU threshold range from 0.50 to 0.95 (with a step size of 0.05), providing a more thorough assessment of the model's detection performance at various overlap levels.

In terms of speed, we assess the model's inference speed using FPS (Frames Per Second), with the formula for calculation provided below:

$$FPS = \frac{1000}{a + b + c} \tag{24}$$

where $a$ represents the preprocessing time, $b$ denotes the time required for model inference, and $c$ refers to the time needed for non-maximum suppression.

Additionally, we assess the model's complexity using several evaluation metrics, including the number of parameters, computational complexity, and model size. These factors collectively reflect the overall complexity of the model.

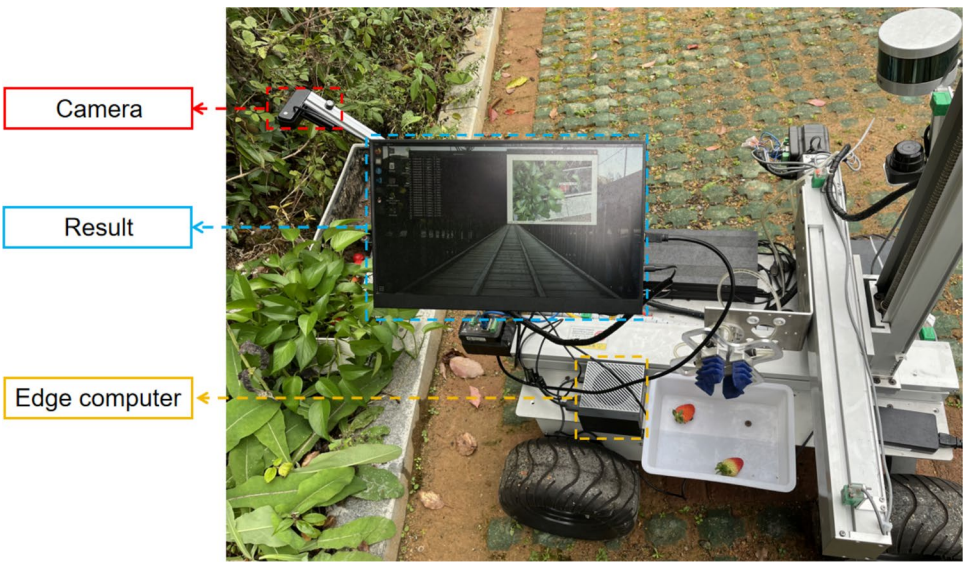


**Fig. 11** Deploying the SR-YOLO detection model on the high-performance Jetson AGX Orin edge computing platform for the real-time strawberry fruit detection application

## Comparisons of different datasets

To validate the reliability of our improved model, we conducted comprehensive evaluations on four strawberry datasets (LSDS, StrawDI_Db1, Straw_Rf, and Straw_Sc) and compared its performance against the baseline model, YOLO11n. Detailed results are presented in Table 4. On the public dataset, StrawDI_Db1, our model showed notable improvements in precision and mAP50, with increases of 2.5% and 2.1%, respectively. These improvements can be attributed to the high-resolution images, which preserve more detailed information. Similarly, on the open-source dataset, Straw_Rf, our model demonstrated substantial gains in recall and accuracy. Precision increased from 85.2% to 87.4%, and recall rose from 78.6% to 80.3%. These enhancements may be due to the simplicity of the dataset, where each image typically contains only a few strawberry fruits, making target identification easier for the model. On our self-cultivated dataset, Straw_Sc, the mAP50 also improved by 1.5%, further validating the model's effectiveness. Finally, on the most complex and data-rich LSDS dataset, our model continued to perform excellently, with precision and mAP50 improving by 2.8% and 2.5%, respectively. Overall, the SR-YOLO model outperformed the baseline YOLO11n across key performance indicators such as precision, recall, and mAP. While there was a slight decrease in FPS, this can likely be attributed to the computational burden of the extensive use of the ELA attention mechanism. In conclusion, the improved model is capable of replacing the baseline model and can be deployed in real-time strawberry target detection across various maturity stages in complex environments.

## Comparison of different feature fusion structures

To validate the superiority of the lightweight LAFPN proposed in this study, we compared the baseline model, based on the YOLO11n algorithm and equipped with PAN-FPN, against several advanced feature pyramid structures, including BiFPN, HS-FPN, CARAFE-FPN, and AFPN, using the LSDS dataset. The experimental results are presented in Table 5. The incorporation of BiFPN, CARAFE-FPN, and AFPN into the baseline model significantly improved performance across various metrics. Specifically, the model with BiFPN showed a 0.6% increase in precision. Additionally, CARAFE-FPN led to a substantial improvement in mAP50, increasing from 89.6% to 90.4%. However, this improvement came at the cost of an additional 0.3 M parameters and a 0.6G increase in computational complexity, which makes it less suitable for deployment on embedded devices for real-time strawberry detection. In contrast, the introduction of AFPN resulted in substantial improvements in key performance indicators. precision and mAP50 improved by 1.3% and 0.7%, respectively, with a reduction in the number of parameters—an important factor for resource-constrained applications. Notably, although HS-FPN reduced model parameters by 0.3 M and computational complexity by 0.7G, recall decreased by 0.6%, potentially leading to missed detections of strawberries. This suggests that HS-FPN may overlook crucial detailed features, such as the color, shape, and position of strawberries, due to its emphasis on channel attention. Ultimately, LAFPN demonstrated exceptional performance in strawberry maturity detection. It reduced the number of parameters by 0.1 M and FLOPs by 0.2G

**Table 4** Performance comparison of SR-YOLO and YOLO11n on four strawberry datasets

| Dataset | Models | Precision | Recall | mAP50 | mAP95 | FPS |
|---|---|---|---|---|---|---|
| LSDS | Baseline | 88.5 | 83.8 | 89.6 | 73.8 | 107.5 |
| | Improved | 90.9 | 85.3 | 92.1 | 75.0 | 102.4 |
| StrawDI_Db1 | Baseline | 90.7 | 87.0 | 92.4 | 77.6 | 101.1 |
| | Improved | 93.2 | 88.2 | 94.5 | 78.6 | 95.8 |
| Straw_Rf | Baseline | 85.2 | 78.6 | 83.1 | 67.2 | 95.2 |
| | Improved | 87.4 | 80.3 | 85.9 | 68.3 | 91.6 |
| Straw_Sc | Baseline | 80.8 | 77.0 | 82.0 | 60.9 | 93.5 |
| | Improved | 83.3 | 78.4 | 83.5 | 62.2 | 89.7 |

**Table 5** Performance comparison of different feature pyramid structures on the LSDS dataset

| Feature fusion structures | Precision | Recall | mAP50 | mAP95 | Param.(M) | FLOPs(G) |
|---|---|---|---|---|---|---|
| Baseline | 88.5 | 83.8 | 89.6 | 73.8 | 2.6 | 6.3 |
| BiFPN [30] | 89.1 | 83.9 | 89.9 | 74.0 | 2.6 | 6.3 |
| CARAFE-FPN [16] | 89.4 | 84.7 | 90.4 | 74.2 | 2.9 | 6.9 |
| HS-FPN [31] | 88.9 | 83.2 | 89.4 | 73.6 | **2.3** | **5.6** |
| AFPN [25] | 89.8 | 84.5 | 90.3 | 74.1 | 2.4 | 5.9 |
| Ours(LAFPN) | **90.1** | **84.7** | **90.8** | **74.4** | 2.5 | 6.1 |

while improving mAP50 and mAP95 by 1.2% and 0.6%, respectively. Compared to other advanced feature pyramid modules, the superiority of the LAFPN model in enhancing overall detection accuracy is evident. These results clearly demonstrate the exceptional ability of the LAFPN model to capture fine-grained features in strawberry images and highlight its advanced approach to feature fusion.

To assess the effectiveness of the ELA module embedded in the feature localization stage of the LAFPN structure, we used the Grad-CAM (Gradient-weighted Class Activation Mapping) method (Selvaraju et al., 2017) to generate visual heat maps. These heat maps illustrate the areas of the image that the model focuses on, with the intensity of the red hue reflecting the level of influence these regions have on target localization. This visualization technique allows us to intuitively analyze the contributions of the ELA module to model performance, as shown in Fig. 12. In Fig. 12a-c, we present the heat maps for models without the ELA module. These heat maps show that the model tends to focus on non-strawberry areas, such as the leaves surrounding the strawberries or the background. This misdirected attention could lead to misjudgments or missed detections, particularly when strawberries are occluded or set against complex backgrounds. In these cases, the model may overlook the actual target due to excessive attention on irrelevant background details. In contrast, Fig. 12d-f show the heat maps of models equipped with the ELA module. These heat maps clearly demonstrate that the model now concentrates on the strawberry target regions, even in cases where parts of the strawberry are blurred or occluded. The ELA module enhances edge information and local features, enabling the model to more precisely locate strawberries. This enhancement decreases the chances of false positives and false negatives, enabling more robust and accurate target recognition, even in challenging or occluded scenarios.

## Ablation study

### IoU loss function

To assess the effectiveness of the proposed loss function module in improving detection performance, we conducted ablation experiments on the LSDS strawberry dataset using YOLO11n as the baseline. As shown in Table 6, replacing the original CIoU loss function with the Focaler-IoU loss function resulted in a slight decrease in FPS, but significantly improved the model's precision and mAP50, with increases of 0.2% and 0.3%, respectively. This suggests that considering the varying difficulty of samples during regression effectively enhances the model's adaptability

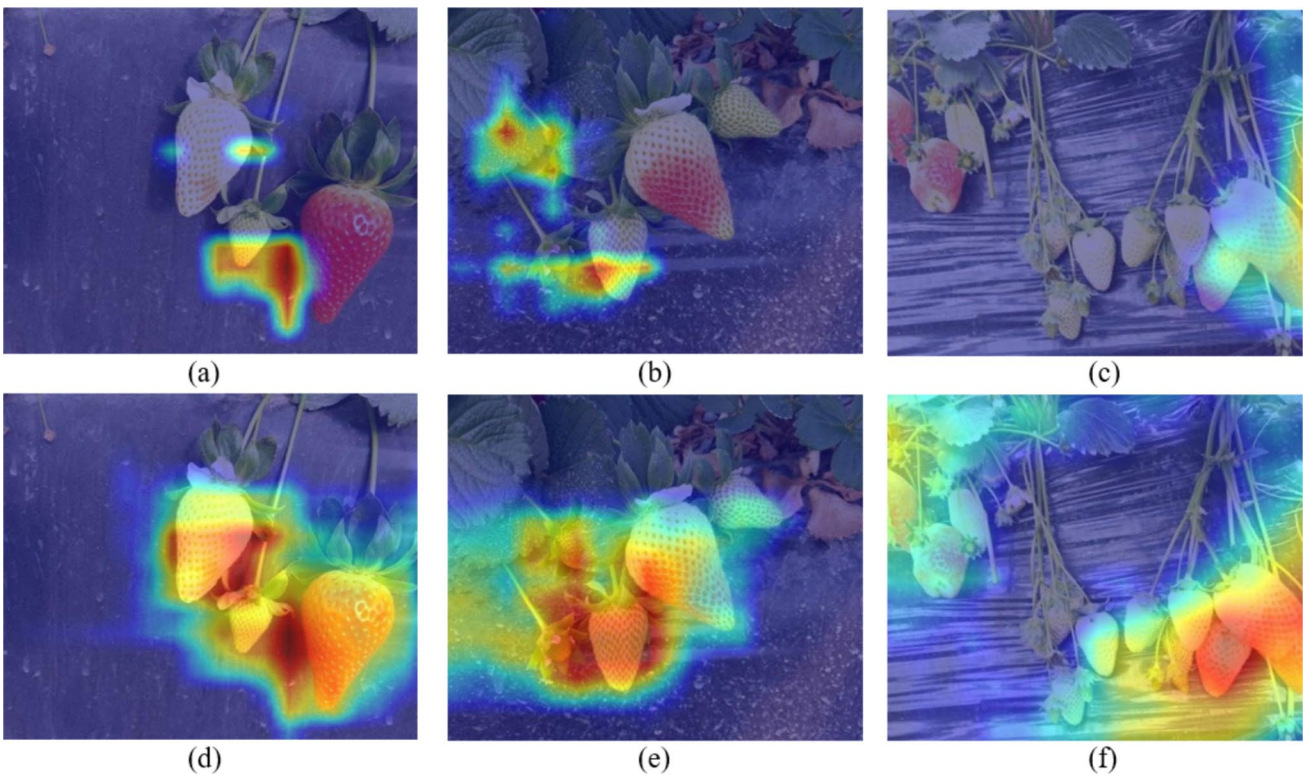


**Fig. 12** Comparison of Grad-CAM heat maps for strawberry fruit detection without (**a**-c) and with (**d**-**f**) the ELA module embedded in the neck network

and detection capabilities for strawberry detection. Further analysis revealed that replacing the CIoU loss function with the PIoU loss function led to more notable improvements in recall and mAP50, which increased by 0.4% and 0.5%, respectively. Notably, when Focaler-IoU was combined with PIoU to form Focaler-PIoU, the model achieved its best performance: mAP50 rose from 89.6% to 90.3%, and FPS increased by 3.6. This not only enhanced the model's accuracy in detecting challenging samples but also accelerated its convergence speed. These results clearly demonstrate that the introduction of PIoU and Focaler-IoU significantly boosts both the accuracy and efficiency of the detection model. Moreover, they confirm the superior performance of the Focaler-PIoU loss function in the task of strawberry ripeness detection.

To further validate the effectiveness of our designed Focaler-PIoU loss function, we analyzed the loss curves on both the training and validation sets. The loss curve on the training set is shown in Fig. 13a, and the loss curve on the validation set is shown in Fig. 13b. Both the Focaler-PIoU and PIoU loss functions exhibited faster convergence speeds. This can be attributed to the PIoU loss function guiding the anchor boxes to regress along a more effective path, significantly accelerating the model's convergence compared to the traditional CIoU loss. These results strongly indicate that PIoU, Focaler-IoU, and our proposed Focaler-PIoU loss function all effectively reduce loss values, thereby significantly enhancing detection performance.

### Overall strategies

To assess the impact of the proposed modules, we performed a series of ablation experiments on the LSDS dataset, as detailed in Table 7. Initially, substituting the C2f bottleneck with the Pela block resulted in an increase in recall rate from 83.8% to 84.3%, and FPS improved by 13.1%. This shows

**Table 6** Ablation study on the impact of loss functions on model performance

| CIoU | Focaler-IoU | PIoU | Precsion | Recall | mAP50 | mAP95 | FPS |
|---|---|---|---|---|---|---|---|
| √ | | | 88.5 | 83.8 | 89.6 | 73.8 | 107.5 |
| | | √ | 88.9 | **84.2** | 90.1 | 74.1 | 109.9 |
| | √ | | 88.7 | 83.9 | 89.9 | 74.0 | 105.3 |
| | √ | √ | **89.2** | **84.2** | **90.3** | **74.2** | **111.1** |

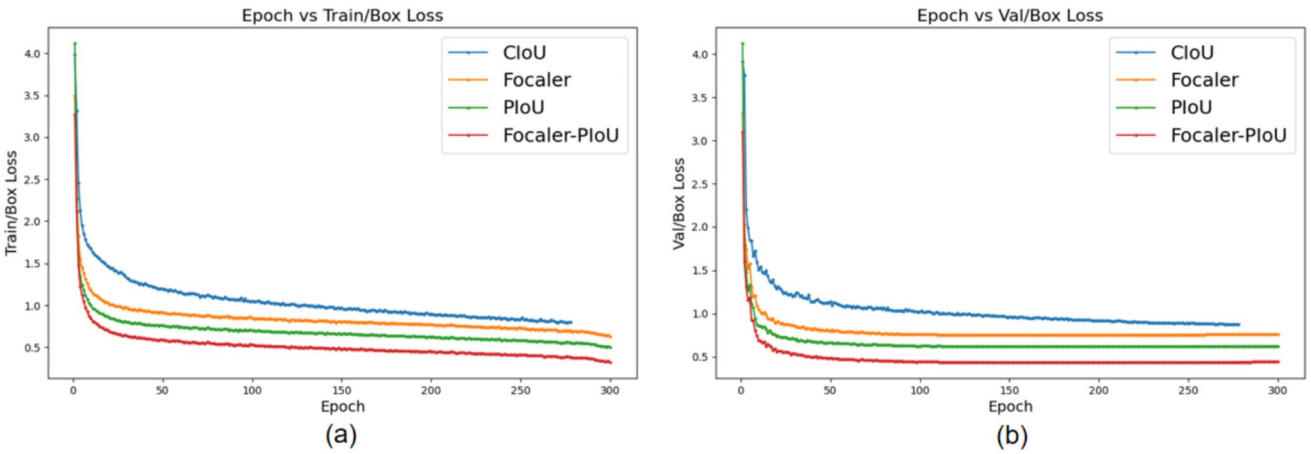


**Fig. 13** Comparison of loss curves from ablation experiments with different IoU loss functions on (**a**) the training set and (**b**) the validation set

**Table 7** Ablation study on the impact of proposed modules on model performance. ① represents Pela block, ② represents LAFPN, and ③ represents Focaler-PIoU

| ① | ② | ③ | Precision | Recall | mAP50 | mAP95 | Param.(M) | FLOPs(G) | FPS |
|---|---|---|---|---|---|---|---|---|---|
| | | | 88.5 | 83.8 | 89.6 | 73.8 | 2.6 | 6.3 | 107.5 |
| √ | | | 89.0 | 84.3 | 90.1 | 74.2 | 2.4 | 6.0 | **121.6** |
| √ | √ | | 90.7 | 85.1 | 91.5 | 74.7 | **2.3** | **5.8** | 98.7 |
| √ | | √ | 89.5 | 84.7 | 90.7 | 74.5 | 2.4 | 6.0 | 125.8 |
| | √ | √ | 90.1 | 85.0 | 91.4 | 74.7 | 2.5 | 6.1 | 86.3 |
| √ | √ | √ | **91.3** | **85.3** | **92.1** | **75.0** | **2.3** | **5.8** | 102.4 |

that the Pela block greatly improves the model's ability to locate strawberries at different stages of ripeness, while improving computational efficiency and processing speed. Next, adopting the LAFPN model as the neck network further reduced the number of model parameters and FLOPs, while increasing precision and mAP50 by 1.7% and 1.4%, respectively. This improvement compensates for the loss of fine-grained information in the feature pyramid fusion process and reduces computational complexity and memory requirements, enhancing the model's suitability for deployment on resource-limited portable devices. Finally, by introducing the Focaler-PIoU loss function, the model's mAP50 and mAP95 increased by 0.5% and 0.4%, respectively, significantly enhancing detection accuracy in challenging scenarios such as overlapping and occlusion, and accelerating convergence. After integrating all these strategies, our model achieves a 2.8% improvement in precision, a 2.5% increase in mAP50, an 11.1% reduction in model parameters, and a 0.5G decrease in computational complexity, alongside only a slight decrease in FPS. These results clearly demonstrate that our model achieves an optimal balance between precision and speed, and can effectively replace the baseline model for real-time and accurate strawberry detection across various maturity stages in complex real-world environments.

To provide a more detailed comparison of the performance between our model and the baseline in strawberry ripeness detection, we conducted an in-depth analysis. The test results for the baseline model are displayed in Fig. 14a and c. The results of our improved model are presented in Fig. 14b and d. In Fig. 14a, strawberries marked with'u'were not detected by the baseline model due to their small size, highlighting the baseline model's limited ability to extract features from small strawberry targets. Additionally, partially ripe strawberries marked with'v'were missed because of occlusion by stems and leaves, further emphasizing the baseline model's difficulties in handling occlusions. In Fig. 14c, strawberries marked with'w'were detected multiple times due to high overlap with adjacent strawberries. This demonstrates the baseline model's shortcomings in processing overlapping targets. In contrast, our improved model effectively addresses these limitations by incorporating several key strategies. Firstly, by integrating the Pela block, we significantly enhance the model's feature extraction capabilities, allowing it to more accurately distinguish strawberry targets from background interference. Second, the introduction of the LAFPN model greatly improves detection performance for small-sized strawberries, enabling accurate feature extraction even when the strawberries are extremely small. Finally, the application of the Focaler-Shape-IoU technique improves the identification rate of difficult samples in overlapping scenarios. In conclusion, the integration of these techniques allows our model to effectively manage occlusion, detecting small targets, and distinguishing overlapping strawberries, overcoming the baseline model's limitations in complex scenarios.

## Comparisons with state-of-the-art models

To comprehensively evaluate the performance of the proposed model in strawberry ripeness detection, we conducted experiments on the LSDS strawberry dataset and compared our approach against several representative

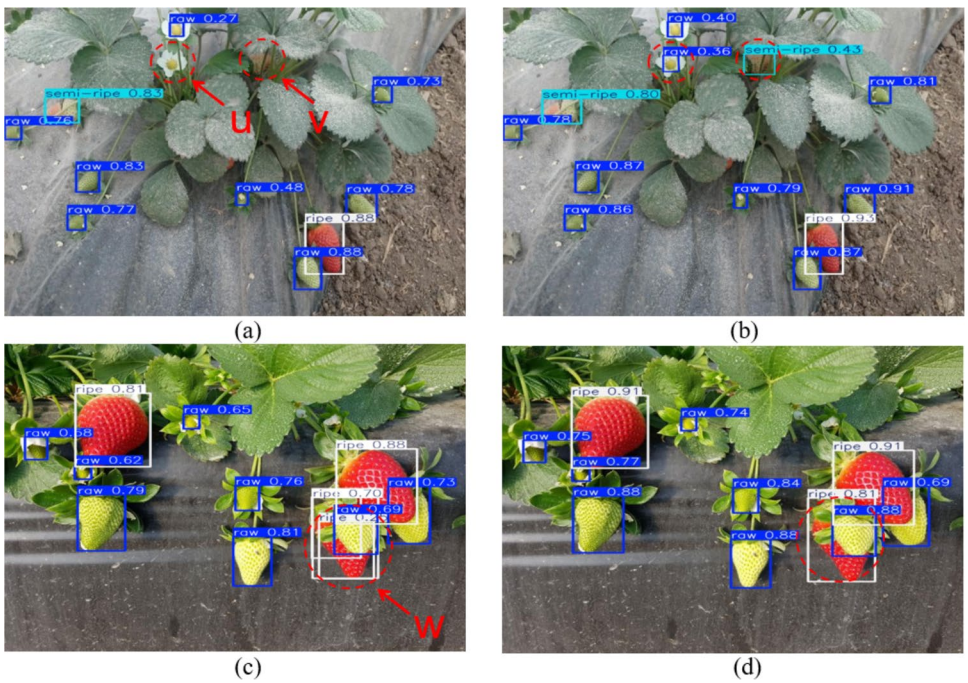


**Fig. 14** Comparison of the strawberry target detection results using our model (**b**) and (**d**) with the baseline model (**a**) and (**c**). The red dashed circles indicate the missed targets or false positives, while the dark blue, light blue, and white rectangles represent the detected raw, semi-ripe, and ripe strawberry targets, respectively

object detection models. The results of the experiments are presented in Table 8. Additionally, we set up a special experiment using YOLO11n as the baseline model and compared it with outstanding lightweight backbone networks such as MobileNetV4, StarNet, FasterNet, and ConvNeXtV2. Specifically, our model achieves a 7.4% higher precision and a 10.0% improvement in mAP50 compared to the EfficientDet-D0 model. This advantage likely stems from EfficientDet's design, which focuses on balancing speed and accuracy, but may encounter performance bottlenecks in small object detection and complex backgrounds. Similarly, Although RTMDet-tiny achieves a mAP50 of 87.2%, its number of parameters is 2.1 times that of our model, resulting in a large model size and slower detection speed, which makes it unsuitable for real-time strawberry detection applications. Notably, the end-to-end network YOLOv10n performs excellently, achieving mAP50 and mAP95 of 89.2% and 73.6%, respectively. However, Our model's computational complexity is reduced to 92.1% of the baseline, making it better suited for deployment on embedded devices. Compared to other single-stage YOLO algorithms, our model also shows significant performance improvements, such as a 3.6% higher mAP50 compared to the classic YOLOv5n and a 3.3% higher mAP50 compared to the popular YOLOv8n. When compared to models using lightweight backbone networks, especially those equipped with MobileNetV4, StarNet, FasterNet, and ConvNeXtV2, our model demonstrates a clear advantage. In particular, on the key metric of mAP50, our model outperforms the best-performing YOLO11n-ConvNeXtV2 by 5.3%. In summary, despite its reduced parameters and lower FLOPs compared to other models, our SR-YOLO achieves exceptional detection performance, making it an ideal solution for real-time, accurate strawberry ripeness detection on edge devices.

Models labeled with a pound sign (#) represent end-to-end networks, those with an ampersand (&) indicate models using different backbone networks, while unmarked models are categorized as single-stage networks.

To demonstrate our model's effectiveness in detecting strawberries at different ripeness stages, we conducted comparative studies on the LSDS dataset with three representative networks: the widely-used YOLO model (YOLOv8n), the end-to-end network YOLOv10n, and the single-stage network YOLO11n-ConvNeXtV2, which combines YOLO11n with the ConvNeXtv2 backbone network. Specifically, as shown in Fig. 15, subfigures (a), (b), (c), and (d) correspond to YOLOv8n, YOLO11n-ConvNeXtv2, YOLOv10n, and our improved model, respectively. Firstly, in Fig. 15a, although YOLOv8n performed well in detecting most strawberries, it failed to recognize small strawberries marked as 'q' and 't'. Additionally, due to overlapping strawberries, YOLOv8n missed the detection of strawberries marked as 'r' and 's'. These results highlight significant shortcomings in YOLOv8n's ability to detect small targets and handle overlapping instances. Next, the YOLO11n-ConvNeXtv2 model, which incorporates the ConvNeXtv2 backbone, performed better at detecting small and overlapping strawberries, as seen in Fig. 15b. However, it missed strawberries marked as 'v', which were similar in color to the surrounding background, and 'w', which were highly occluded. Moreover, YOLO11n-ConvNeXtv2 incorrectly detected the strawberry marked as 'u', indicating that the model still struggles with distinguishing between background and strawberries in complex occluded scenarios. Moving to the YOLOv10n model, as shown in Fig. 15c, it failed to detect the small strawberry marked as 'x' and missed strawberries marked as 'y' and 'z'. This clearly demonstrates YOLOv10n's limitations in detecting small targets. By comparison, our model excelled in Fig. 15d, effectively addressing the issues of small target

**Table 8** Performance comparison of our model with the state-of-the-art ones on the LSDS dataset

| Model | Precision | Recall | mAP50 | mAP95 | Param.(M) | FLOPs(G) |
|---|---|---|---|---|---|---|
| EfficientDet-D0 [30] | 83.9 | 78.6 | 82.1 | 68.2 | 3.9 | 5.2 |
| RTMDet-tiny [32] | 86.1 | 81.2 | 87.2 | 70.9 | 4.8 | 8.1 |
| YOLOv5n [33] | 88.8 | 82.1 | 88.5 | 73.0 | 2.5 | 7.1 |
| YOLOv6n [34] | 87.9 | 82.3 | 88.7 | 72.8 | 4.2 | 11.8 |
| YOLO7-tiny [35] | 88.4 | 82.0 | 89.1 | 73.1 | 6.0 | 13.2 |
| YOLOv8n [36] | 88.1 | 83.2 | 88.8 | 73.5 | 3.0 | 8.1 |
| YOLOv9t (Wang et al., 2024) | 87.6 | 83.4 | 88.9 | 73.6 | 2.0 | 7.7 |
| YOLOv10n$^{\#}$ (Wang et al., 2024) | 88.3 | 83.4 | 89.2 | 73.6 | 2.7 | 8.2 |
| YOLO11n [37] | 88.5 | 83.8 | 89.6 | 73.8 | 2.6 | 6.3 |
| YOLO11n-MobileNetv4$^{\&}$ [38] | 84.1 | 79.8 | 85.1 | 71.7 | 2.2 | 5.3 |
| YOLO11n-StarNet$^{\&}$ [39] | 83.4 | 79.1 | 84.6 | 70.3 | **2.0** | **5.0** |
| YOLO11n-FasterNet$^{\&}$ (Liu et al., 2023) | 85.6 | 80.6 | 85.7 | 71.9 | 2.4 | 5.6 |
| YOLO11n-ConvNeXtV2$^{\&}$ [40] | 87.7 | 80.7 | 86.8 | 72.7 | 2.6 | 5.9 |
| SR-YOLO(Ours) | **91.3** | **85.3** | **92.1** | **75.0** | 2.3 | 5.8 |

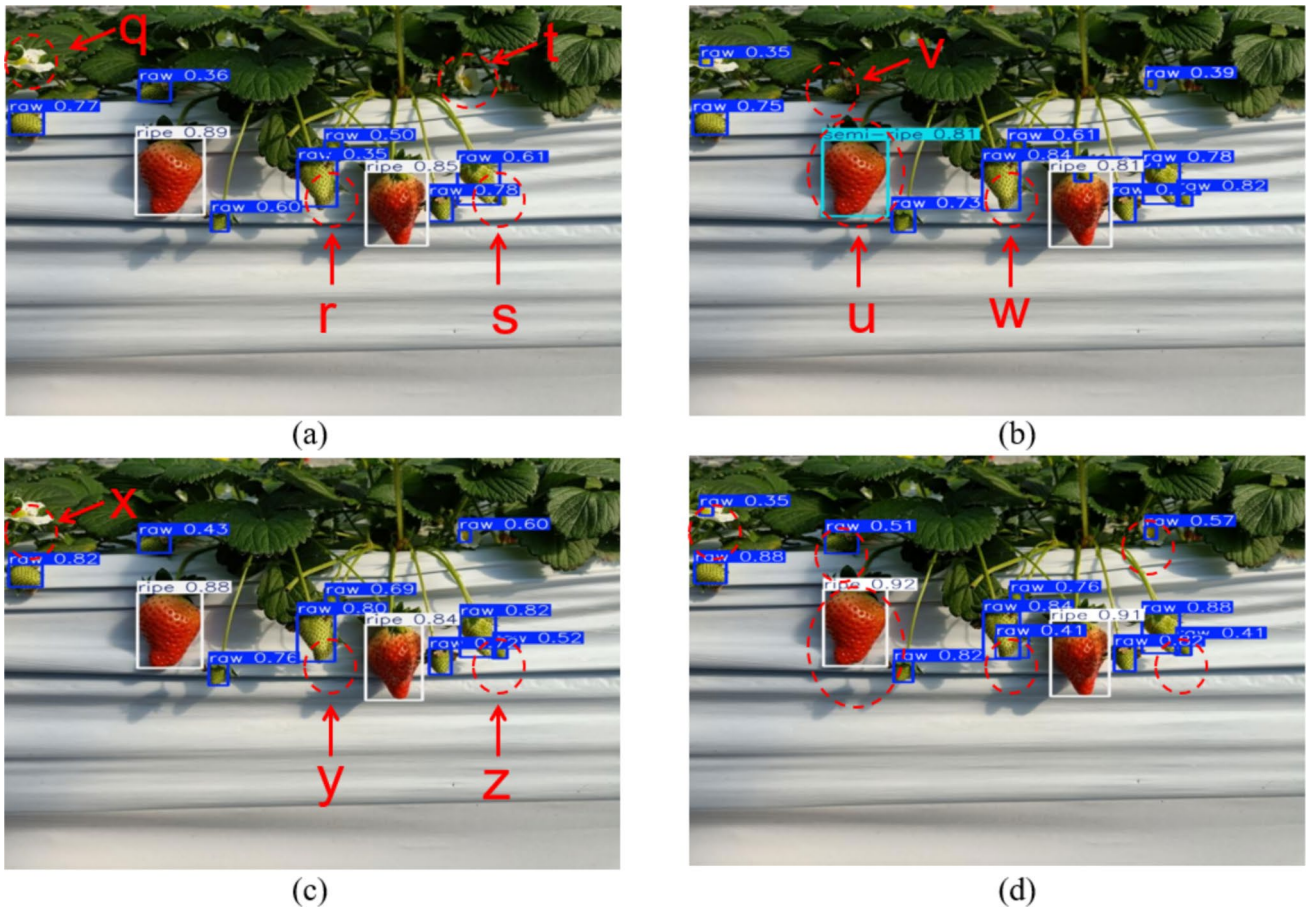


**Fig. 15** Strawberry fruit discrimination results using the state-of-the-art identification frameworks of **a** YOLOv8n, **b** YOLO11n-ConvNeXtV2, **c** YOLOv10n, and **d** SR-YOLO (ours).The red dashed circles indicate the missed targets or false positives, while the dark blue, light blue, and white rectangles represent the detected raw, semi-ripe, and ripe strawberry targets, respectively

detection, overlapping target recognition, and background interference. It accurately detected strawberries across different ripeness stages, successfully recognized small and overlapping targets, and handled background interference with greater precision. These findings demonstrate our model's superior detection accuracy and robustness, particularly in complex real-world scenarios involving occlusion, small targets, and varying lighting conditions.

## Tests on the edge computing platform

To validate the deployment performance of the proposed algorithm, we deployed the improved model on the Jetson AGX Orin edge computing device. It is important to note that, compared to high-end server GPUs, the GPU performance of edge devices is relatively weaker, which can lead to reduced detection accuracy [41]. To address this, we conducted detection speed tests on various models deployed on the Jetson AGX Orin. The models tested include YOLOv5n, YOLOv6n, YOLOv7-tiny, YOLOv8n, YOLOv9t, YOLOv10n, YOLO11n, and our proposed SR-YOLO algorithm. The detection speed (FPS), mAP50, and model size data for these YOLO models on the Jetson AGX Orin are shown in Table 9. As shown in the table, YOLOv10n achieves the highest FPS at 101.2, but its

**Table 9** Comparisons of mAP50, detection speed, and model size utilizing the YOLO-relevant models on Jetson AGX Orin edge computing device

| Models | mAP50 | FPS | Model size(MB) |
|---|---|---|---|
| YOLOv5n | 82.1 | 90.4 | 5.3 |
| YOLOv6n | 82.0 | 78.8 | 8.7 |
| YOLOv7-tiny | 81.8 | 85.4 | 13.2 |
| YOLOv8n | 82.4 | 91.8 | 6.2 |
| YOLOv9t | 82.2 | 82.4 | **4.6** |
| YOLOv10n | 82.9 | **99.2** | 5.7 |
| YOLO11n | 82.6 | 94.4 | 5.5 |
| SR-YOLO | **84.3** | 90.1 | 5.1 |

detection performance is subpar, with an mAP50 of only 82.9%, failing to meet the accuracy requirements for strawberry detection. In comparison, SR-YOLO excels in both detection speed (88.1 FPS) and accuracy (mAP50 of 84.3%), significantly outperforming most other models. Additionally, compared to the baseline model, SR-YOLO has a reduced model size of 0.4 M, enhancing its suitability for deployment on portable devices. In summary, SR-YOLO strikes an ideal balance between detection accuracy and speed, making it particularly well-suited for deployment on edge devices for real-time, precise strawberry detection across different maturity stages.

Furthermore, the power efficiency of the SR-YOLO model is also considered during its deployment on the Jetson AGX Orin platform. As the Orin module supports dynamic power scaling between 15 and 60 W, the inference process of SR-YOLO operates under a 30 W configuration, maintaining a stable detection speed of 90.1 FPS. Preliminary measurements indicate an average power consumption of approximately 22 W during continuous operation. This results in an energy efficiency of 4.1 FPS/W, highlighting the model's suitability for deployment in power-constrained environments. Such efficiency is particularly advantageous for autonomous strawberry-picking robots, where battery life is a critical consideration in field applications.

To better visualize and highlight the superiority of our improved model in terms of detection speed and model size, we used radar charts (Fig. 16a) and line charts (Fig. 16b) to present the FPS and parameter quantity from Table 2. As shown in Fig. 16a, our improved model exhibits superior real-time performance for strawberry detection. In Fig. 16b, the model size of SR-YOLO is second only to YOLOv9t, making it highly advantageous for deployment on embedded devices. It can be seen that, compared with other state-of-the-art models, the proposed SR-YOLO model is smaller in size, higher in operation speed and accuracy, and is quite suitable for deployment on edge computing equipment to detect strawberries in real time across different maturity stages in complex agricultural environments.

## Conclusion

This study introduced a lightweight YOLO11-based model, SR-YOLO, aimed at achieving high-precision real-time strawberry ripeness detection under complex agricultural environments. The proposed Pela block replaces the Bottleneck in the C3k2 module to improve feature discrimination in visually similar backgrounds, while the LAFPN module enhances multi-scale feature extraction. Furthermore, the Focaler-PIoU loss function addresses sample imbalance and improves bounding box regression accuracy. The final model achieves 91.3% precision and 92.1% mAP50 with only 2.3 M parameters and 5.8G FLOPs. Deployed on Jetson AGX Orin, it reaches 90.1 FPS, meeting the real-time demands of edge devices in agricultural robotics.

Nevertheless, several limitations must be acknowledged. Firstly, the three-stage classification of ripeness adopted in this study simplifies the naturally continuous maturation process of strawberries, which may lead to misclassification near stage boundaries. Secondly, while the model was trained on a large and diverse dataset, its generalization may be limited under extreme agricultural conditions, such as heavy rain, dense fog, strong backlighting, or non-standard cultivation environments. These factors could affect image quality and introduce significant noise, thereby reducing detection robustness. To address these limitations, future work will focus on two main directions: (1) developing a more fine-grained and continuous ripeness classification system that better reflects the full strawberry growth cycle, and (2) expanding the dataset to include samples from a broader range of environmental conditions, strawberry cultivars,

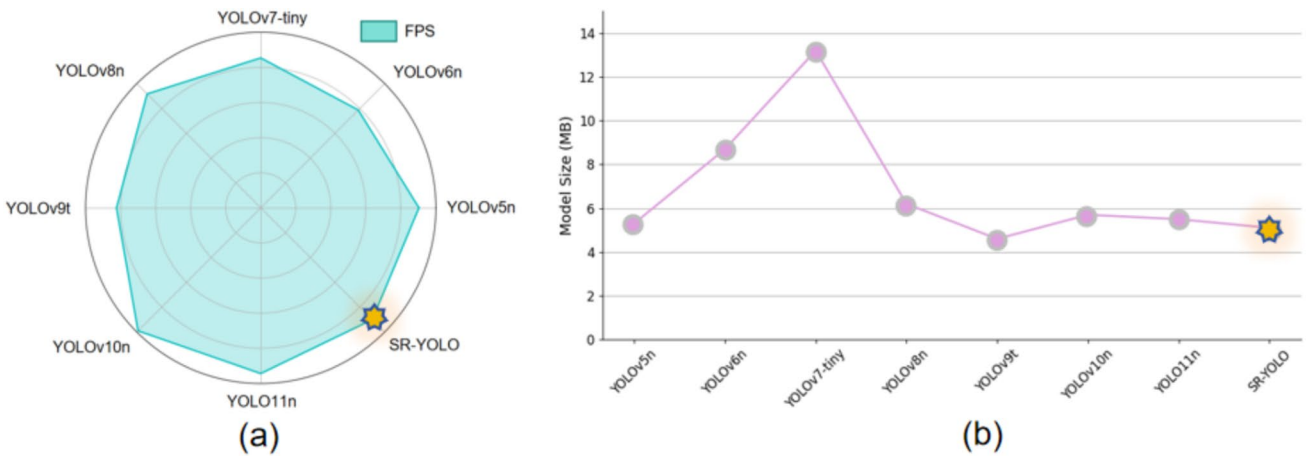


**Fig. 16** Comparisons of detection speed (**a**) and model size (**b**) for SR-YOLO with other state-of-the-art models

and growth settings. These improvements are expected to enhance the model's adaptability, robustness, and generalization across real-world, high-variability scenarios.

**Acknowledgements** This study was supported by the National Natural Science Foundation of China (Grant No. 31601227).

**Data Availability** All strawberry datasets used in this study are available upon reasonable request by contacting ganyang0720@gmail.com.

## Declarations

**Competing interest** The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References

1. N. Tamrakar, S. Karki, M.Y. Kang, N.C. Deb, E. Arulmozhi, D.Y. Kang, J. Kook, H.T. Kim, Lightweight improved YOLOv5s-CGhostnet for detection of strawberry maturity levels and counting. Agriengineering **6**, 962–978 (2024)
2. X. Du, H. Cheng, Z. Ma, W. Lu, M. Wang, Z. Meng, C. Jiang, F. Hong, DSW-Yolo: a detection method for ground-planted strawberry fruits under different occlusion levels. Comput. Electron. Agric. **214**, 108304 (2023)
3. S. Shen, F. Duan, Z. Tian, C. Han, A novel deep learning method for detecting strawberry fruit. Appl. Sci. **14**, 4213 (2024)
4. S.A. Mostafa, S. Ravi, D.A. Zebari, N.A. Zebari, M.A. Mohammed, J. Nedoma, R. Martinek, M. Deveci, W. Ding, A YOLO-based deep learning model for real-time face mask detection via drone surveillance in public spaces. Inf. Sci. (2024). https://doi.org/10.1016/j.ins.2024.120865
5. J.K. Basak, B.G.K. Madhavi, B. Paudel, N.E. Kim, H.T. Kim, Prediction of total soluble solids and pH of strawberry fruits using RGB, HSV and HSL colour spaces and machine learning models. Foods **11**, 2086 (2022)
6. P. Constante, A. Gordon, O. Chang, E. Pruna, F. Acuna, I. Escobar, Artificial vision techniques to optimize strawberry's industrial classification. IEEE Lat. Am. Trans. **14**, 2576–2581 (2016)
7. Y. Fitter, Strawberry detection under various harvestation stages. arXiv (2019)California Polytechnic State University.
8. Y. Bai, J. Yu, S. Yang, J. Ning, An improved YOLO algorithm for detecting flowers and fruits on strawberry seedlings. Biosyst. Eng. **237**, 1–12 (2024)
9. LeCun, Y., Kavukcuoglu, K., Farabet, C. In Convolutional networks and applications in vision, Proceedings of 2010 IEEE international symposium on circuits and systems, IEEE, pp 253–256 (2010)
10. W. Ji, Y. Pan, B. Xu, J. Wang, A real-time apple targets detection method for picking robot based on ShufflenetV2-YOLOX. Agriculture **12**, 856 (2022)
11. C. Yang, J. Liu, J. He, A lightweight waxberry fruit detection model based on YOLOv5. IET Image Process. **18**, 1796–1808 (2024)
12. Q. Sun, P. Li, C. He, Q. Song, J. Chen, X. Kong, Z. Luo, A lightweight and high-precision passion fruit YOLO detection model for deployment in embedded devices. Sensors **24**, 4942 (2024)
13. Y. Wang, G. Yan, Q. Meng, T. Yao, J. Han, B. Zhang, DSE-YOLO: Detail semantics enhancement YOLO for multi-stage strawberry detection. Comput. Electron. Agric. **198**, 107057 (2022)
14. F. Pang, X. Chen, MS-yolov5: a lightweight algorithm for strawberry ripeness detection based on deep learning. Syst. Sci. Control Eng. **11**, 2285292 (2023)
15. S. Yang, W. Wang, S. Gao, Z. Deng, Strawberry ripeness detection based on YOLOv8 algorithm fused with LW-Swin transformer. Comput. Electron. Agric. **215**, 108360 (2023)
16. H. Liu, X. Wang, F. Zhao, F. Yu, P. Lin, Y. Gan, X. Ren, Y. Chen, J. Tu, Upgrading swin-B transformer-based model for accurately identifying ripe strawberries by coupling task-aligned one-stage object detection mechanism. Comput. Electron. Agric. **218**, 108674 (2024)
17. K. He, X. Zhang, S. Ren, J. Sun, In Deep residual learning for image recognition, Proceedings of the IEEE conference on computer vision and pattern recognition, pp 770–778 (2016)
18. J. Chen, S.-h. Kao, H. He, W. Zhuo, S. Wen, C.-H. Lee, S.-H.G. Chan, In Run, don't walk: chasing higher FLOPS for faster neural networks, Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 12021–12031 (2023)
19. W. Xu, Y. Wan, ELA: Efficient local attention for deep convolutional neural networks. arXiv:2403.01123, (2024)
20. Q. Luo, C. Wu, G. Wu, W. Li, A small target strawberry recognition method based on improved YOLOv8n model. IEEE Access (2024). https://doi.org/10.1109/ACCESS.2024.3356869
21. C. Louizos, M. Welling, D.P. Kingma, Learning sparse neural networks through $ L_0 $ regularization. arXiv:1712.01312, (2017)
22. S. Liu, L. Qi, H. Qin, H. Shi, J. Jia, In Path aggregation network for instance segmentation, Proceedings of the IEEE conference on computer vision and pattern recognition, pp 8759–8768 (2018)
23. J. Liu, B. Kang, C. Liu, X. Peng, Y. Bai, YOLO-BFRV: an efficient model for detecting printed circuit board defects. Sensors **24**, 6055 (2024)
24. X. Liu, Y. Wang, D. Yu, Z. Yuan, YOLOv8-FDD: a real-time vehicle detection method based on improved YOLOv8. IEEE Access (2024). https://doi.org/10.1109/ACCESS.2024.3453298
25. G. Yang, J. Lei, Z. Zhu, S. Cheng, Z. Feng, R. Liang, In AFPN: Asymptotic feature pyramid network for object detection, 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, pp 2184–2189 (2023)
26. S. Liu, D. Huang, Y. Wang, Learning spatial fusion for single-shot object detection. arXiv (2019)arXiv:2404.10518.
27. W. Liu, H. Lu, H. Fu, Z. Cao, In Learning to upsample by learning to sample, Proceedings of the IEEE/CVF International Conference on Computer Vision, pp 6027–6037 (2023)
28. C. Liu, K. Wang, Q. Li, F. Zhao, K. Zhao, H. Ma, Powerful-IoU: more straightforward and faster bounding box regression loss with a nonmonotonic focusing mechanism. Neural Netw. **170**, 276–284 (2024)
29. H. Zhang, S. Zhang, Focaler-IoU: More focused intersection over union loss. arXiv:2401.10525, (2024)
30. M. Tan, R. Pang, Q.V. Le, In Efficientdet: Scalable and efficient object detection, Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 10781–10790 (2020)
31. Y. Chen, C. Zhang, B. Chen, Y. Huang, Y. Sun, C. Wang, X. Fu, Y. Dai, F. Qin, Y.JCi.B. Peng, Accurate leukocyte detection based on deformable-DETR and multi-level feature fusion for aiding diagnosis of blood diseases. Medicine **170**, 107917 (2024)
32. C. Lyu, W. Zhang, H. Huang, Y. Zhou, Y. Wang, Y. Liu, Y. Zhang, K. Chen, Rtmdet: An empirical study of designing real-time object detectors. arXiv (2022)arXiv:2212.07784.
33. G. Jocher, A. Chaurasia, A. Stoken, J. Borovec, Y. Kwon, J. Fang, K. Michael, D. Montes, J. Nadar, P. Skalski, ultralytics/yolov5: v6. 1-tensorrt, tensorflow edge tpu and openvino export and inference. Zenodo, (2022)
34. C. Li, L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie, YOLOv6: A single-stage object detection framework for industrial applications. arXiv (2022)arXiv:2209.02976.

35. C.-Y. Wang, A. Bochkovskiy, H.-Y.M. Liao, In YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 7464–7475 (2023)
36. G. Jocher, In Ultralytics yolov8. https://github.com/ultralytics/ultralytics, (2023)
37. G. Jocher, In Ultralytics yolo11. https://github.com/ultralytics/ultralytics, (2024)
38. D. Qin, C. Leichner, M. Delakis, M. Fornoni, S. Luo, F. Yang, W. Wang, C. Banbury, C. Ye, B. Akin, MobileNetV4-Universal Models for the Mobile Ecosystem. arXiv (2024)arXiv:2404.10518.
39. X. Ma, X. Dai, Y. Bai, Y. Wang, Y. Fu, In rewrite the stars, proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp 5694–5703 (2024)
40. S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. Kweon, S. Xie, V2: Co-Designing and Scaling ConvNets with Masked Autoencoders. arXiv:2301.00808, (2023)
41. Q. Chen, X. Jiang, A portable real-time concrete bridge damage detection system. Measurement **240**, 115536 (2025)